DATA STRUCTURES

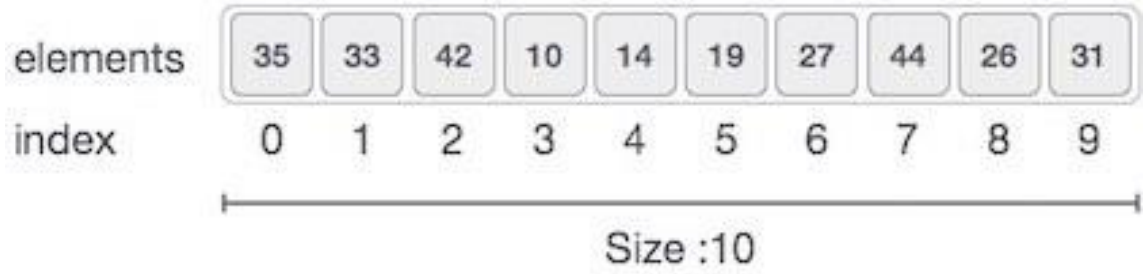
UNIT - 2

Array

- Array is a container which can hold a fix number of items and these items should be of the same type. Most of the data structures make use of arrays to implement their algorithms. Following are the important terms to understand the concept of Array.
- **Element** – Each item stored in an array is called an element.
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element.
- **Properties of array**
- There are some of the properties of an array that are listed as follows -
- Each element in an array is of the same data type and carries the same size.
- Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Array Representation

- Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



Array Representation

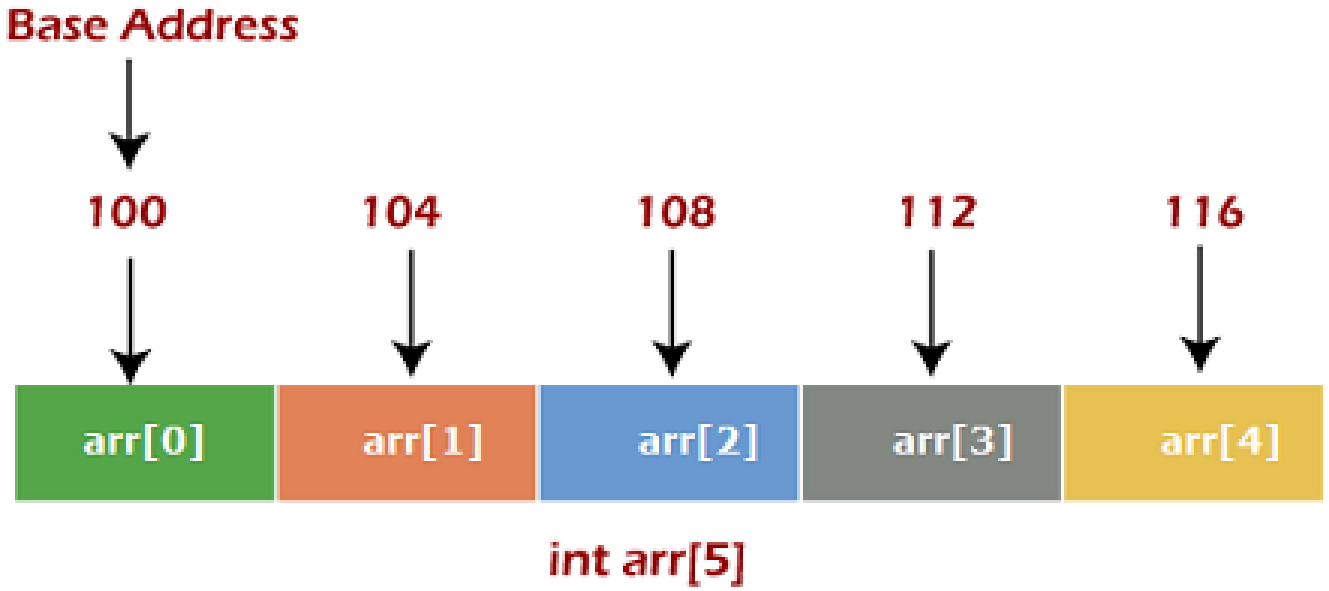
- As per the above illustration, following are the important points to be considered.
- Index starts with 0.
- Array length is 10 which means it can store 10 elements.
- Each element can be accessed via its index. For example, we can fetch an element at index 6 is 27.

Why are arrays required?

- Arrays are useful because -
- Sorting and searching a value in an array is easier.
- Arrays are best to process multiple values quickly and easily.
- **Arrays are good for storing multiple values in a single variable -** In computer programming, most cases require storing a large number of data of a similar type. To store such an amount of data, we need to define a large number of variables. It would be very difficult to remember the names of all the variables while writing the programs. Instead of naming all the variables with a different name, it is better to define an array and store all the elements into it.

Memory allocation of an array

- As stated above, all the data elements of an array are stored at contiguous locations in the main memory. The name of the array represents the base address or the address of the first element in the main memory. Each element of the array is represented by proper indexing.
- We can define the indexing of an array in the below way -
- The first element of the array will be arr[0].

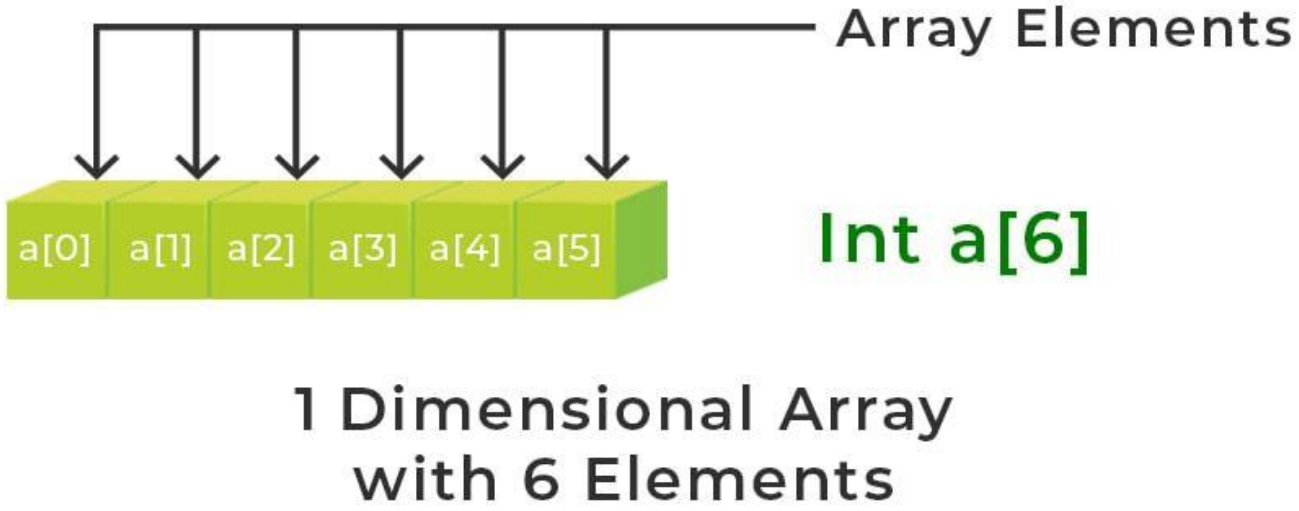


Memory allocation of an array

- In the above image, we have shown the memory allocation of an array arr of size 5. The array follows a 0-based indexing approach. The base address of the array is 100 bytes. It is the address of arr[0]. Here, the size of the data type used is 4 bytes; therefore, each element will take 4 bytes in the memory.

Calculating the address of any element In the 1-D array:

- A 1-dimensional array (or single-dimension array) is a type of linear array. Accessing its elements involves a single subscript that can either represent a row or column index.
- **Example:**



Calculating the address of any element In the 1-D array:

- To find the address of an element in an array the following formula is used-
  Address of A[I] = B + W * (I – LB)
- *I = Subset of element whose address to be found,*
  B = Base address,
  W = Storage size of one element store in any array(in byte),
  LB = Lower Limit/Lower Bound of subscript(If not specified assume zero).
- **Example:** Given the base address of an array **A[1300 …… 1900]** as **1020** and the size of each element is 2 bytes in the memory, find the address of **A[1700].**
- ***Given:***
  Base address B = 1020
  Lower Limit/Lower Bound of subscript LB = 1300
  Storage size of one element store in any array W = 2 Byte
  Subset of element whose address to be found I = 1700
- ***Formula used:***
  *Address of A[I] = B + W * (I – LB)*
- ***Solution:***
  *Address of A[1700] = 1020 + 2 * (1700 – 1300)*
  *= 1020 + 2 * (400)*
  = 1020 + 800
  Address of A[1700] = 1820

Calculate the address of any element in the 2-D array:

- The 2-dimensional array can be defined as an array of arrays. The 2-Dimensional arrays are organized as matrices which can be represented as the collection of rows and columns as array[M][N] where M is the number of rows and N is the number of columns.

Columns

Rows

2D Array	0	1	2
0	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]
2	a[2][0]	a[2][1]	a[2][2]

Calculate the address of any element in the 2-D array:

- To find the address of any element in a **2-Dimensional** array there are the following two ways-
- Row Major Order
- Column Major Order

Calculate the address of any element in the 2-D array:

- **1. Row Major Order:**
- Row major ordering assigns successive elements, moving across the rows and then down the next row, to successive memory locations. In simple language, the elements of an array are stored in a Row-Wise fashion.
- arr[3][3] =
  [a00, a01, a02]
  [b10, b11, b12]
  [c20, c21, c22]
- Row major order = a00, a01, a02, b10, b11, b12, c20, c12, c22

Calculate the address of any element in the 2-D array:

- **1. Row Major Order:**
- ***Address of A[I][J] = B + W * ((I – LR) * N + (J – LC))***
- *I = Row Subset of an element whose address to be found,*
  J = Column Subset of an element whose address to be found,
  B = Base address,
  W = Storage size of one element store in an array(in byte),
  LR = Lower Limit of row/start row index of the matrix(If not given assume it as zero),
  LC = Lower Limit of column/start column index of the matrix(If not given assume it as zero),
  N = Number of column given in the matrix.

Calculate the address of any element in the 2-D array:

- **Example: Example:** Given an array, **arr[1……10][1……15]** with base value **100** and the size of each element is **1 Byte** in memory. Find the address of **arr[8][6]** with the help of row-major order.
- ***Given:***
  Base address B = 100
  Storage size of one element store in any array W = 1 Bytes
  Row Subset of an element whose address to be found I = 8
  Column Subset of an element whose address to be found J = 6
  Lower Limit of row/start row index of matrix LR = 1
  Lower Limit of column/start column index of matrix = 1
  Number of column given in the matrix N = Upper Bound – Lower Bound + 1
  = 15 – 1 + 1
  = 15
- ***Formula:***
  *Address of A[I][J] = B + W * ((I – LR) * N + (J – LC))*
- ***Solution:***
  *Address of A[8][6] = 100 + 1 * ((8 – 1) * 15 + (6 – 1))*
  *= 100 + 1 * ((7) * 15 + (5))*
  *= 100 + 1 * (110)*
  Address of A[I][J] = 210

Calculate the address of any element in the 2-D array:

- **2. Column Major Order:**
- If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order.
- arr[3][3] =
  [a00, a01, a02]
  [b10, b11, b12]
  [c20, c21, c22]
- Column major order = a00, b10, c20, a01, b11, c21, a02, b12, c22

Calculate the address of any element in the 2-D array:

- **2. Column Major Order:**To find the address of the element using column-major order use the following formula:
- ***Address of A[I][J] = B + W * ((J – LC) * M + (I – LR))***
- *I = Row Subset of an element whose address to be found,*
  J = Column Subset of an element whose address to be found,
  B = Base address,
  W = Storage size of one element store in any array(in byte),
  LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),
  LC = Lower Limit of column/start column index of matrix(If not given assume it as zero),
  M = Number of rows given in the matrix.

Calculate the address of any element in the 2-D array:

- **Example:** Given an array **arr[1……10][1……15]** with a base value of **100** and the size of each element is **1 Byte** in memory find the address of arr[8][6] with the help of column-major order.
- ***Given:***
  Base address B = 100
  Storage size of one element store in any array W = 1 Bytes
  Row Subset of an element whose address to be found I = 8
  Column Subset of an element whose address to be found J = 6
  Lower Limit of row/start row index of matrix LR = 1
  Lower Limit of column/start column index of matrix = 1
  Number of column given in the matrix M = Upper Bound – Lower Bound + 1
  = 10 – 1 + 1
  = 10
- ***Formula: used***
  *Address of A[I][J] = B + W * ((J – LC) * M + (I – LR))*
  *Address of A[8][6] = 100 + 1 * ((6 – 1) * 10 + (8 – 1))*
  *= 100 + 1 * ((5) * 10 + (7))*
  *= 100 + 1 * (57)*
  Address of A[I][J] = 157

Calculate the address of any element in the 2-D array:

- From the above examples, it can be observed that for the same position two different address locations are obtained that’s because in row-major order movement is done across the rows and then down to the next row, and in column-major order, first move down to the first column and then next column. So both the answers are right.
- So it’s all based on the position of the element whose address is to be found for some cases the same answers is also obtained with row-major order and column-major order and for some cases, different answers are obtained.

Basic operations array

- Now, let's discuss the basic operations supported in the array -
- Traversal - This operation is used to print the elements of the array.
- Insertion - It is used to add an element at a particular index.
- Deletion - It is used to delete an element from a particular index.
- Search - It is used to search an element using the given index or by the value.
- Update - It updates an element at a particular index.

Traversal operation

- This operation is performed to traverse through the array elements. It prints all array elements one after another. We can understand it with the below program –

```
#include <stdio.h>
void main() {
  int Arr[5] = {18, 30, 15, 70, 12};
int i;
  printf("Elements of the array are:\n");
  for(i = 0; i<5; i++) {
    printf("Arr[%d] = %d,  ", i, Arr[i]);
  }
}
```

- output

```
Elements of the array are:
Arr[0] = 18,  Arr[1] = 30,  Arr[2] = 15,  Arr[3] = 70,  Arr[4] = 12,
```

Insertion operation

- This operation is performed to insert one or more elements into the array. As per the requirements, an element can be added at the beginning, end, or at any index of the array. Now, let's see the implementation of inserting an element into the array.
- Steps
- 1. input array
- 2. input element to be inserted and position at which element will be inserted
- 4. increase the size of array element by 1. // n++
- 3. shift all the elements forward from n-1 to position
- 4. insert element at position
- 5. display array after insertion

```
Array elements before insertion
18 30 15 70 12
Array elements after insertion
18 30 15 50 70 12
```

```c
// C Program to Insert an element at a specific position
#include <stdio.h>
int main()
{
   int arr[100] = { 0 };
   int i, x, pos, n = 5;

   // initial array of size 10
   for (i = 0; i < 5; i++)
      arr[i] = i + 1;

   // print the original array
   for (i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");

   // element to be inserted
   x = 50;

   // position at which element
   // is to be inserted
   pos = 5;

   // increase the size by 1 because we are inserting one new element into array
   n++;

   // shift elements forward
   for (i = n - 1; i >= pos; i--)
      arr[i] = arr[i - 1];

   // insert x at pos
   arr[pos - 1] = x;

   // print the updated array
   for (i = 0; i < n; i++)
      printf("%d ", arr[i]);
   printf("\n");

   return 0;
}
```

Deletion operation

- As the name implies, this operation removes an element from the array and then reorganizes all of the array elements.
- Output

```
Given array elements are :
arr[0] = 18,  arr[1] = 30,  arr[2] = 15,  arr[3] = 70,  arr[4] = 12,
Elements of array after deletion:
arr[0] = 18,  arr[1] = 30,  arr[2] = 15,  arr[3] = 70,
```

```
#include <stdio.h>
#include <conio.h>

void main()
{
   int i, n, index, arr[10];

   printf("Enter the size of the array: ");
   scanf("%d", &n);
   printf("Enter the elements of the array: \n");
   for (i = 0; i < n; i++)
   {
      printf("arr[%d] = ", i);
      scanf("%d", &arr[i]);
   }
   printf("Enter the index of the element to be deleted: ");
   scanf("%d", &index);
   if (index >= n+1)
   {
      printf (" \n Deletion is not possible in the array.");
   }
   else
   {
      for (i = index; i < n - 1; i++) // starts replacing from delete element to n-1
                  arr[i] = arr[i + 1]; //shift array elements
     printf("The array after deleting the element is: ");
      for (i = 0; i < n - 1; i++)
         printf("%d ", arr[i]);

   }
   getch(); }
```

Search operation

- This operation is performed to search an element in the array based on the value or index.
- Output

```
Given array elements are :
arr[0] = 18,  arr[1] = 30,  arr[2] = 15,  arr[3] = 70,  arr[4] = 12,
Element to be searched = 70
Element 70 is found at 4 position
```

```
#include <stdio.h>
#include <conio.h>
int main()
{
   int a[10000],i,n,ele;

   printf("Enter size of the  array : ");
   scanf("%d", &n);
   printf("Enter elements in array : ");
   for(i=0; i<n; i++)
   {

      scanf("%d",&a[i]);
   }
   printf("Enter the element : ");
   scanf("%d", &ele);
   for(i=0; i<n; i++)
   {
      if(a[i]==ele)
      {
                printf("element found");

      }

   }
  getch();
}
```

Update operation

- This operation is performed to update an existing array element located at the given index.
- Output

```
Given array elements are :
arr[0] = 18,  arr[1] = 30,  arr[2] = 15,  arr[3] = 70,  arr[4] = 12,
Array elements after updation :
arr[0] = 18,  arr[1] = 30,  arr[2] = 50,  arr[3] = 70,  arr[4] = 12,
```

```
#include <stdio.h>

void main() {
  int arr[5] = {18, 30, 15, 70, 12};
  int item = 50, i, pos = 3;

  printf("Given array elements are :\n");

  for(i = 0; i<5; i++) {
    printf("arr[%d] = %d,  ", i, arr[i]);
  }

arr[pos-1] = item;
  printf("\nArray elements after updation :\n");

  for(i = 0; i<5; i++) {
    printf("arr[%d] = %d,  ", i, arr[i]);
  }
  getch();
}
```

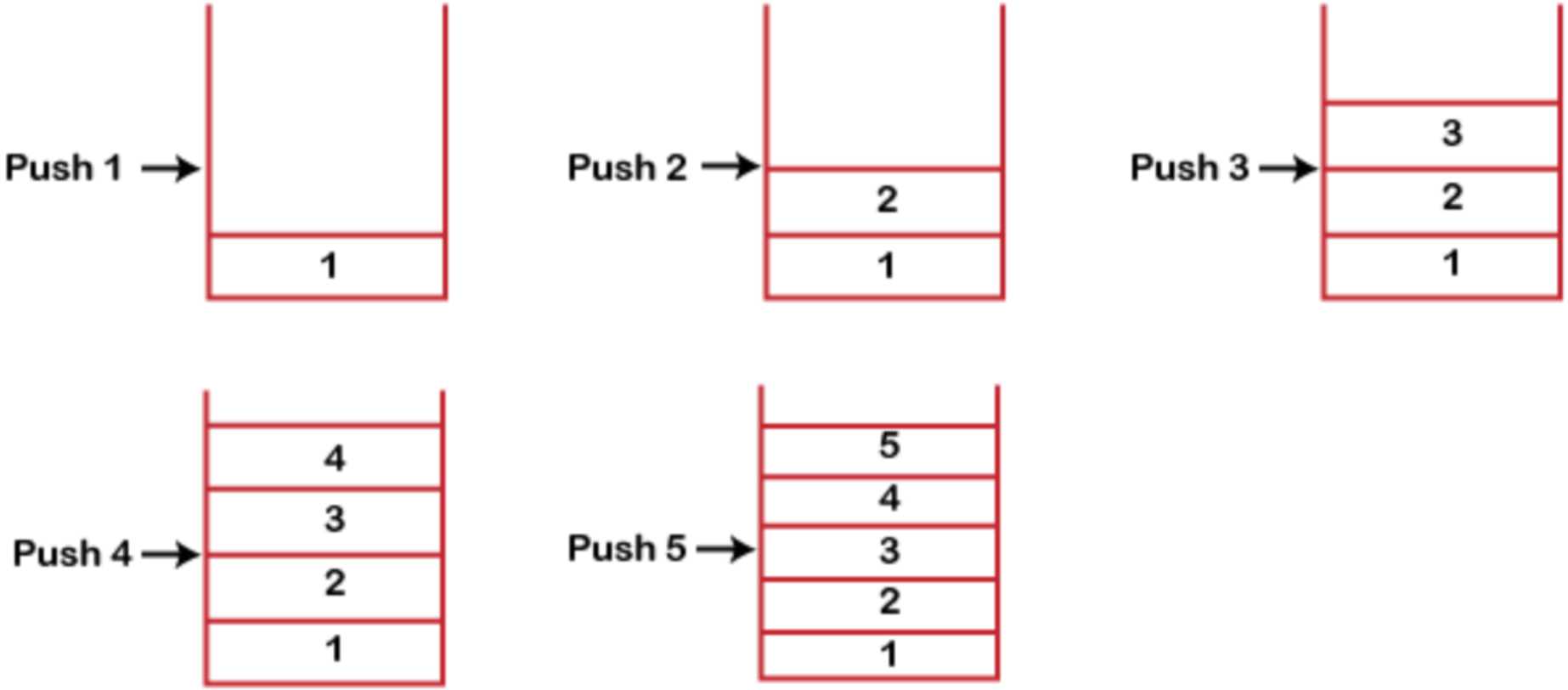
STACK

- A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle.
- Stack has one end, whereas the Queue has two ends (**front and rear**). It contains only one pointer **top pointer** pointing to the topmost element of the stack.
- Whenever an element is added in the stack, it is added on the top of the stack, and the element can be deleted only from the stack.
- In other words, a ***stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.***

STACK

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
- A Stack is an abstract data type.
- It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO.
- Note: An abstract data type is an abstraction of a data structure that provides only the interface to which the data structure must adhere. The interface does not give any specific details about something should be implemented or in what programming language. Abstract Data Type(ADT) is a data type, where only behavior is defined but not implementation. Array, Queue, Stack etc. are examples of ADT.
- **Real life example:** book is Abstract (Telephone Book is an implementation)

STACK



STACK - OPERATIONS

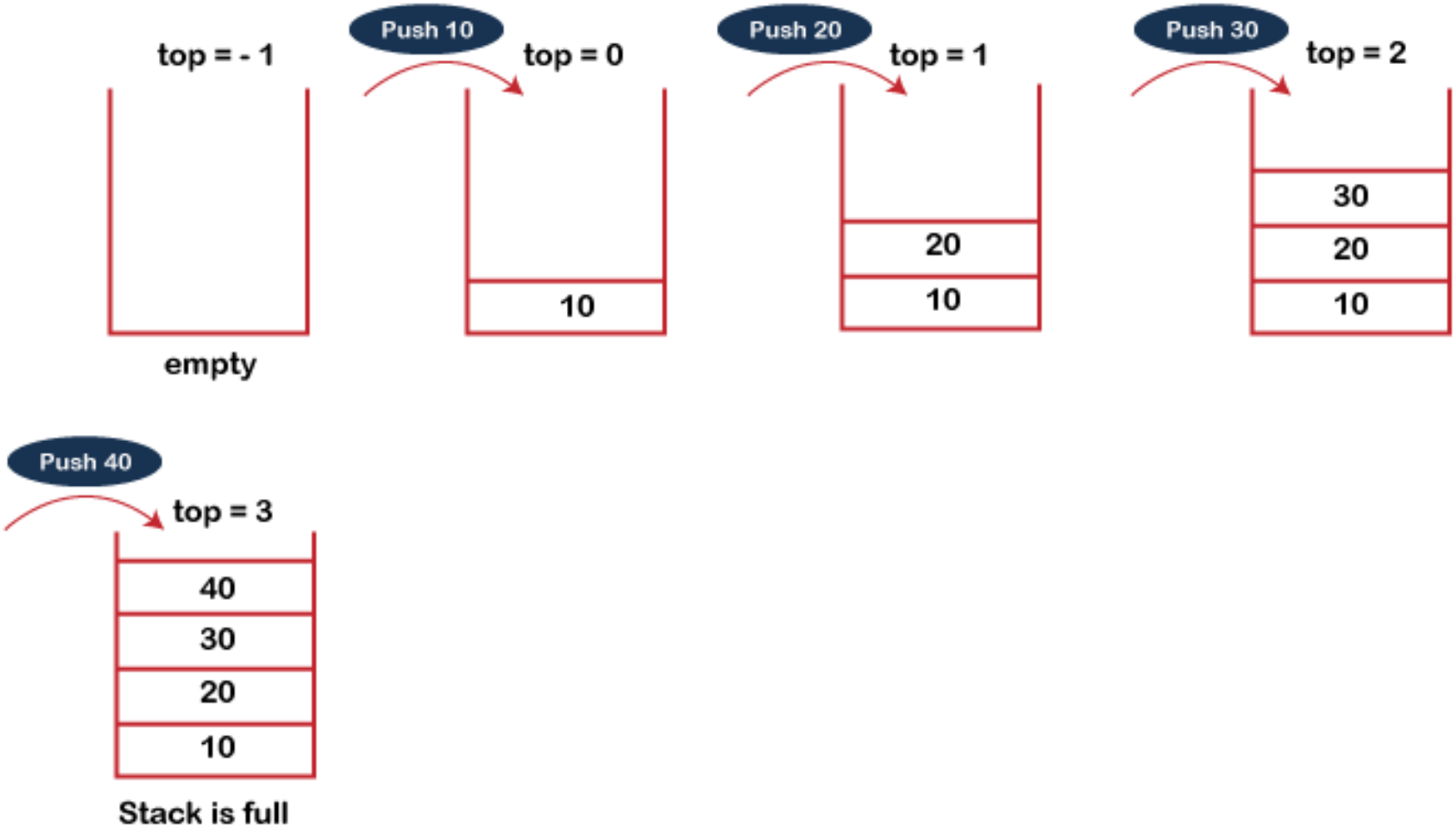
- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **peek():** Returns the top most element.
- **Note: pop() will remove and return the item from the stack while peek() will only return the top most item from the stack it does not remove item from the stack**
- Now to create stack we need to allocate space to stack. So two ways to allocate space: static (array) and dynamic (linked list)
- So we can implement stack using array or linked list

STACK - PUSH

The steps involved in the PUSH operation is given below:

- Before inserting an element in a stack, we check whether the stack is full.
- If we try to insert the element in a stack, and the stack is full, then the ***overflow*** condition occurs.
- When we initialize a stack, we set the value of top as -1 to check that the stack is empty.
- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1,** and the element will be placed at the new position of the **top**.
- The elements will be inserted until we reach the ***max*** size of the stack.

PUSH


top = - 1
empty
Push 10
top = 0
10
Push 20
top = 1
20
10
Push 30
top = 2
30
20
10
Push 40
top = 3
40
30
20
10
Stack is full

ALGORITHM - PUSH

PUSH(STACK, TOP, N, ITEM)

This procedure inserts an item in the stack.

Step 1: [Stack already filled]

If TOP = N-1 then

Print: OVERFLOW

Exit.

Step 2:Set TOP:= TOP + 1 [Increments TOP by 1]

Step 3: Set STACK[TOP] = ITEM [Inserts item in stack]

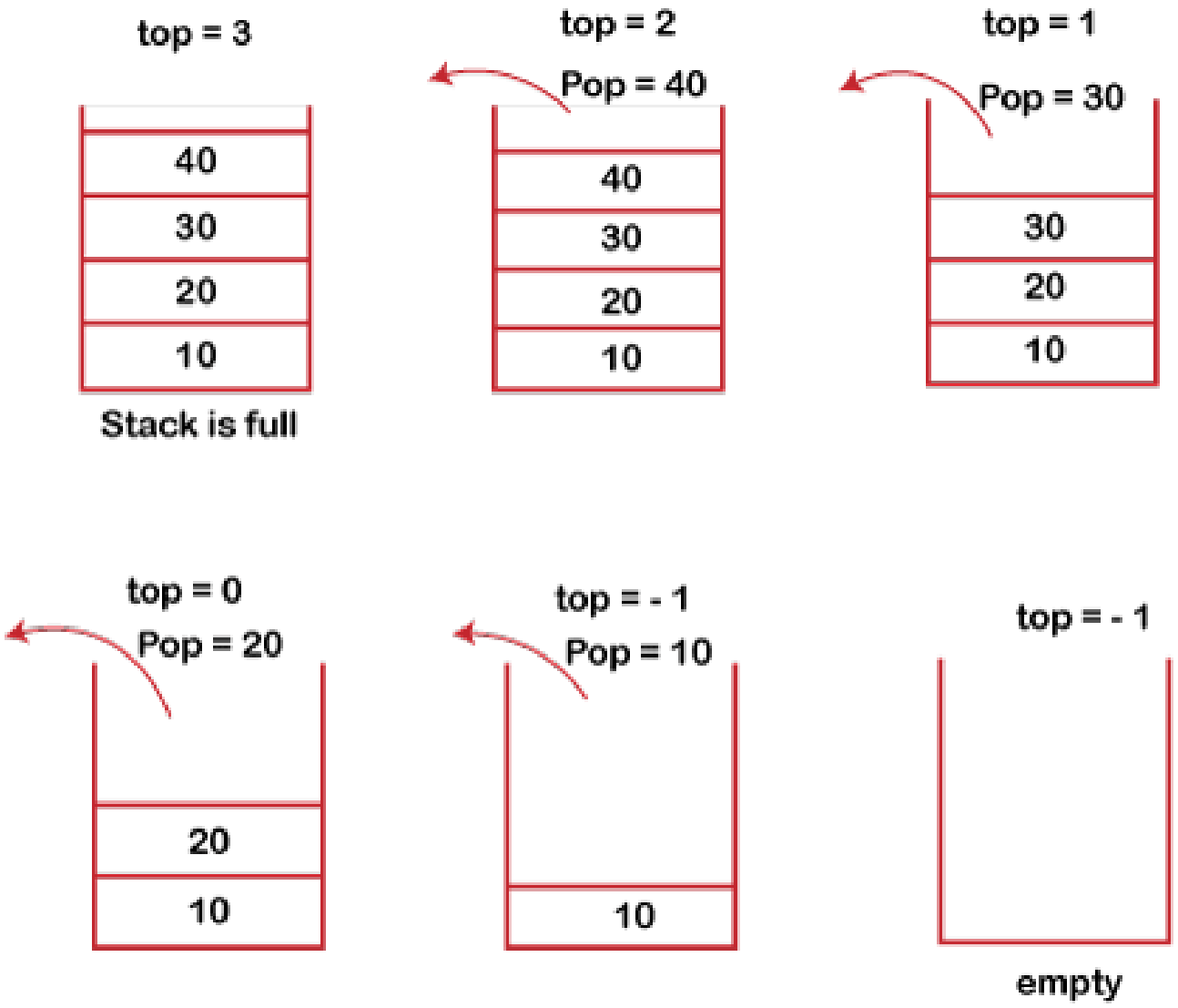
Step 4: Return

STACK - POP

The steps involved in the POP operation is given below:

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the ***underflow*** condition occurs.
- If the stack is not empty, we first access the element which is pointed by the ***top***
- Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1**.

STACK - POP


top = 3
40
30
20
10
Stack is full
top = 2
Pop = 40
40
30
20
10
top = 1
Pop = 30
30
20
10
top = 0
Pop = 20
20
10
top = - 1
Pop = 10
10
top = - 1
empty

ALGORITHM – POP

POP(STACK, TOP, ITEM)

This procedure deletes the top item from the stack.

Step 1: [Stack empty?]

```
        If TOP = -1 then
            Print: UNDERFLOW
            Exit.
```

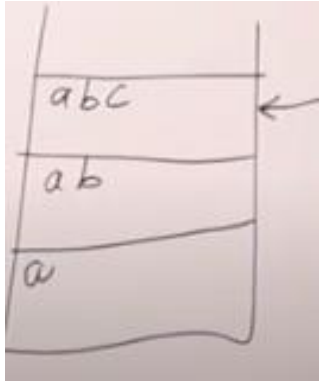
Step 2: Set Item:= STACK[TOP]

Step 3:Set TOP:= TOP - 1 [Decrements TOP by 1]

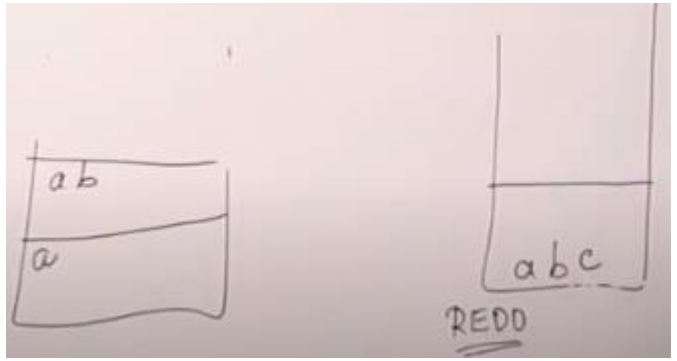
Step 4: Return

STACK - APPLICATIONS

- **String reversal: book (push) -> (pop)koob**
- **UNDO/REDO:** It can also be used for performing UNDO/REDO operations. For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc. So, there are three states, a, ab, and abc, which are stored in a stack. There would be two stacks in which one stack shows UNDO state, and the other shows REDO state. If we want to perform UNDO operation, and want to achieve 'ab' state, then we implement pop operation. Separate stack for undo and redo can be maintained.



Undo



Undo redo

STACK - APPLICATIONS

- **Recursion:** The recursion means that the function is calling itself again. To maintain the previous states, the compiler creates a system stack in which all the previous records of the function are maintained.
- **Expression conversion:** Stack can also be used for expression conversion. This is one of the most important applications of stack.
- Infix, postfix and prefix evaluation

Polish Notation

Polish Notation in the data structure is a method of expressing mathematical, logical, and algebraic equations universally.

This notation is used by the compiler to evaluate mathematical equations based on their order of operations.

When parsing mathematical expressions, three types of notations are commonly used :

1) Infix Notation (Operand Operator Operand)

A + B

2) Postfix notation (Operand Operand Operator)

A B +

3) Prefix Notation (Operator Operand Operand)

+ A B

Polish Notation

Infix Notation :
This polish notation in data structure states that the operator is written in between the operands.

It is the most common type of notation we generally use to represent expressions.

It's the fully parenthesized notation.

We find it much easier to write mathematical expressions in Infix Notation, but it is difficult to parse expressions on computers in the form of infix Polish Notation.

(3+7)

(1*(2+3))

Polish Notation

Prefix Notation :
This polish notation in data structure states that the operator should be present as a prefix or before the operands.

This notation is also known as **"Polish Notation"**.

For example, if we have an expression like x+y, then here x and y are operands, and ‘+’ is the operator.

The prefix notation or polish notation of this expression will be "+xy".

3+7 will convert into +37

1*(2+3) will convert into *1(+23)

Polish Notation

Postfix Notation :
This notation states that the operator should be present as a suffix, postfix, or after the operands.

It is also known as Suffix notation or Reverse Polish Notation. For example, if we have an expression like x+y, then here x and y are operands, and ‘+’ is the operator.

The prefix notation or polish notation of this expression will be "xy+".

Polish Notation

In general, a computer can easily understand postfix expressions.

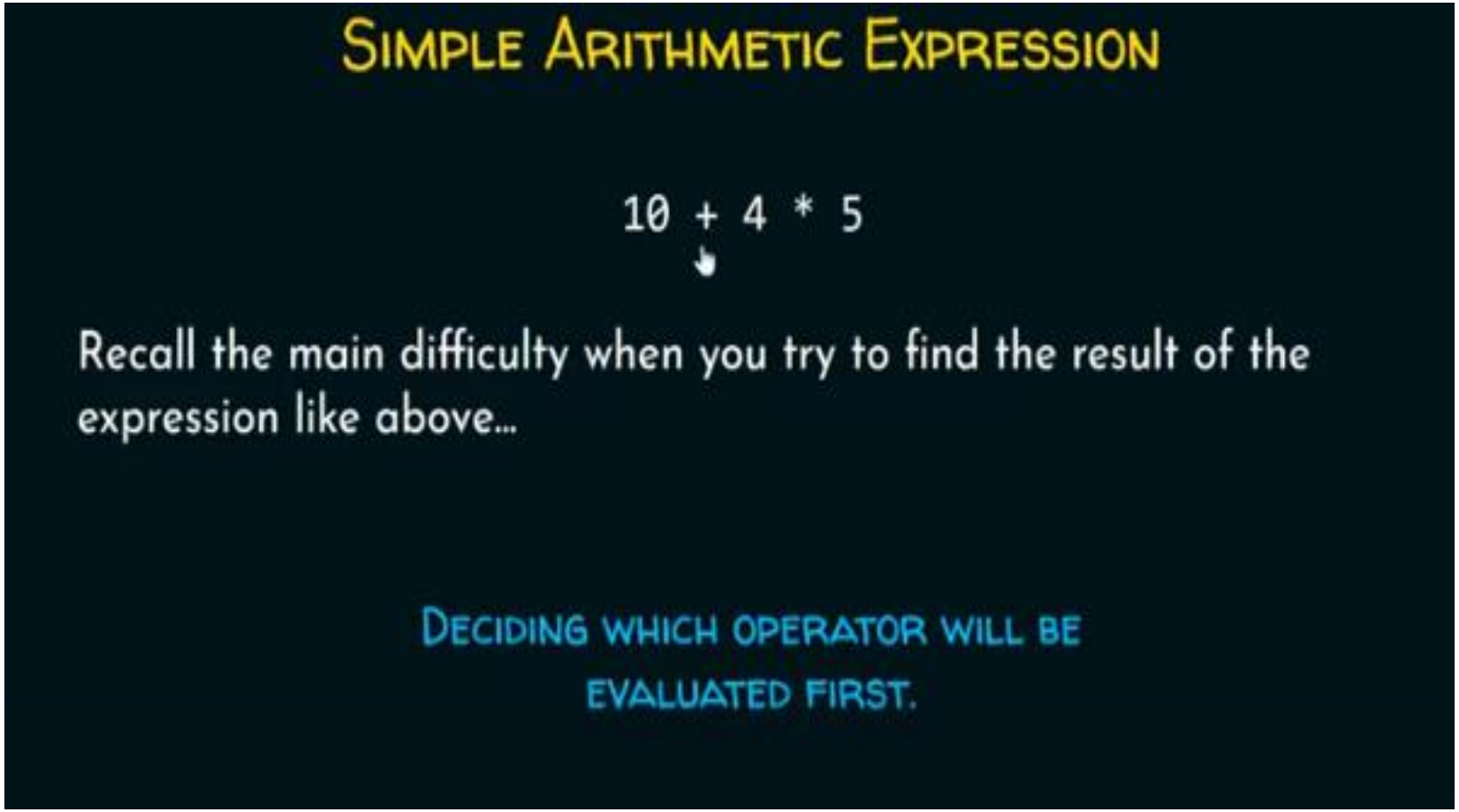
This notation is universally accepted and is preferred for designing and programming the arithmetic and logical units of a **CPU** (Central Processing Unit).

All the expressions that are entered into a computer are converted into **Postfix** or **Reverse Polish Notation**, stored in a stack, and then computed.

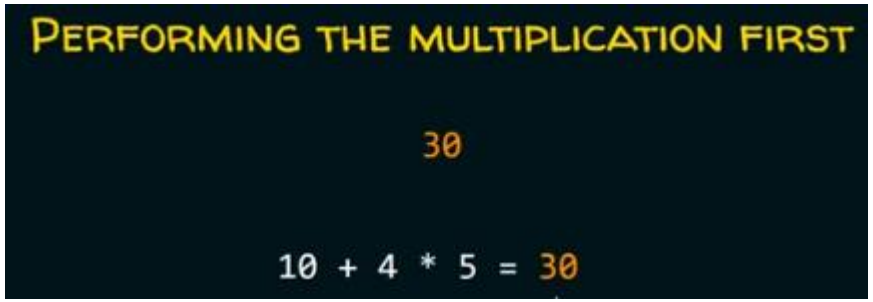
Therefore, postfix expression plays a vital role in the tech industry.

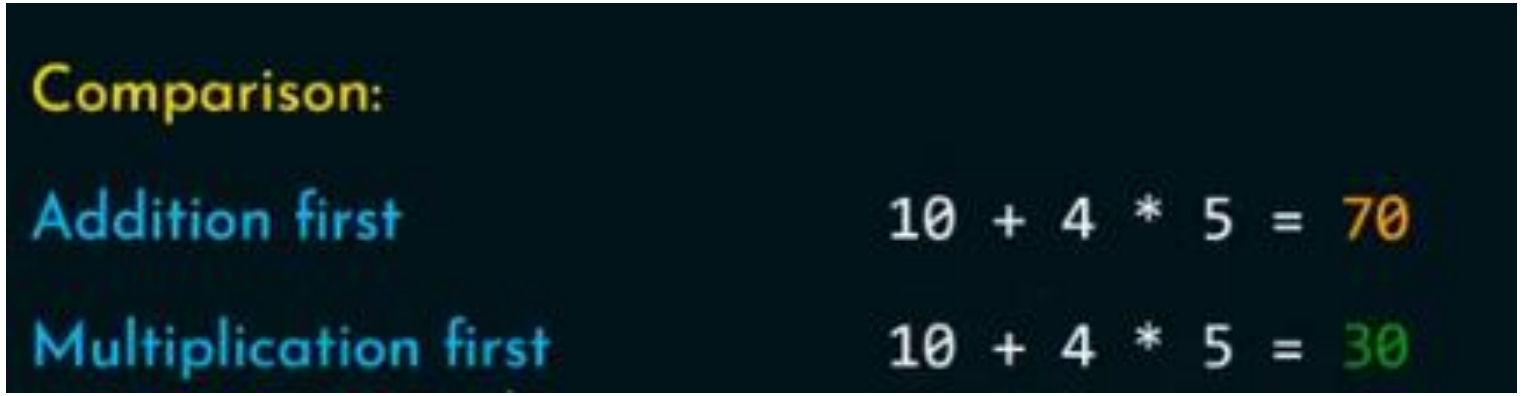
3+7 will convert into 37+

Conversion of an Infix Expression to Postfix Expression



Conversion of an Infix Expression to Postfix Expression





Conversion of an Infix Expression to Postfix Expression

Precedence of operators

Priorities of the operators are predefined.

We will consider the five different operators: +, -, *, /, ^

Operators	Priority
^ (Exponentiation)	1 (Highest)
* (Multiplication) and / (Division)	2
+ (Addition) and - (Subtraction)	3 (Lowest)

Table: Precedence of Operators

Addition first	10 + 4 * 5 = 70	✗
Multiplication first	10 + 4 * 5 = 30	✓

Conversion of an Infix Expression to Postfix Expression

3 + 5 * 7 - 4 ^ 2

3+5*7-16

3+35-16 // + and – have same priority so evaluate from left to right (associativity)

38-16

22

What about the operators with same priority?

3 + 35 - 16

For simplicity, we will assume that the operators with same priority will be evaluated from left to right.

This means whichever operators comes first will be evaluated first.

Conversion of an Infix Expression to Postfix Expression

Find the result of the following infix expression:

```
500 + 5 ^ 3 - (10 + 56) * 9
```

Parentheses have the highest priority

500+5/\ 3 – 66 *9

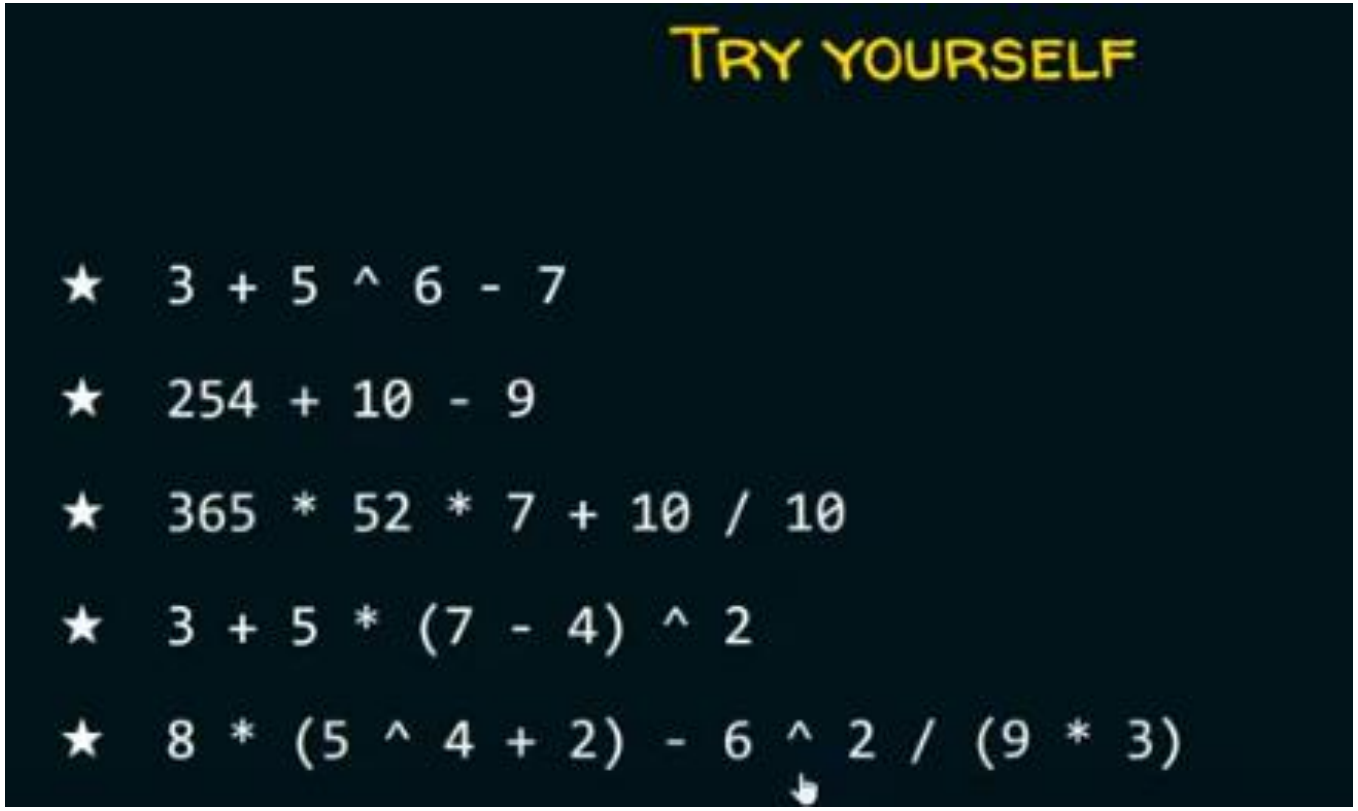
500+125-66*9

500+125-594

625-594

31

Conversion of an Infix Expression to Postfix Expression



Conversion of an Infix Expression to Postfix Expression

- Generally, humans find infix polish notation much easier to understand than postfix or reverse polish notation.
- To convert each expression from infix to postfix, we assign priority to each of the operators present in the expression.
- Each operator has its priority for an expression.
- For example, if we take some operators, i.e., +, -, *, /, then these will be arranged in priority.
- **Higher Priority Operators :** *, /, %.
  Lower Priority Operators : +, -.
  Order of Operators : +, −, ∗, /, ^.

Reverse Polish Notation

- Reverse Polish notation is also known as Postfix notation and Suffix Notation. It plays a vital role in the tech industry.
- In general, humans find Infix polish notation or parenthesized format of expression easy to evaluate, whereas computers find it difficult to parse expressions in the form of Infix Polish Notation, so our computer converts the expression into postfix polish notation or reverse polish notation to evaluate the expression.
- A reverse Polish notation states that the operator should be present after the operands.
- For example, if an expression is x+y, then x and y are operands and **‘+’** is the operator. The prefix notation or polish notation of this expression will be **"xy+"**.

Operator Precedence

( )

↑

*, /

+, -

Algorithm for the conversion from infix to postfix expression

- Algorithm
- 1. First Start scanning the expression from left to right
- 2. If the scanned character is an operand, output it, i.e. print it
- 3. Else
  - If the precedence of the scanned operator is higher than the precedence of the operator in the stack(or stack is empty or has'(‘), then push operator in the stack
  - Else, Pop all the operators, that have greater or equal precedence than the scanned operator. Once you pop them push this scanned operator. (If we see a parenthesis while popping then stop and push scanned operator in the stack)
- 4. If the scanned character is an ‘(‘, push it to the stack.
- 5. If the scanned character is an ‘)’, pop the stack and output it until a ‘(‘ is encountered, and discard both the parenthesis.
- 6. Now, we should repeat the steps 2 – 6 until the whole infix i.e. whole characters are scanned.
- 7. Print output
- 8. Do the pop and output (print) until stack is not empty

Convert from infix to postfix

(A + B) * C

= [A B +] * C

= A B + C *

A + B * C

= A + [B C *]

= A B C * +

(A + B) / ((D - E) * F)

= [A B +] / ([D E -] * F)

= [A B +] / [D E – F *]

= A B + D E – F * /

$$((A + B) / D) \uparrow (E - F * G)$$

$$= ([AB+] / D) \uparrow (E - [FG*])$$

$$= [AB+D/] \uparrow [EFG*-]$$

$$= AB+D/EFG*- \uparrow$$

Infix expression: K + L - M*N + (O^P) * W/U/V * T + Q

Input Expression	Stack	Postfix Expression
K		K
+	+	
L	+	K L
-	-	K L+
M	-	K L+ M
*	- *	K L+ M
N	- *	K L + M N
+	+	K L + M N* K L + M N* -
(	+ (	K L + M N *-
O	+ (	K L + M N * - O
^	+ (^	K L + M N* - O
P	+ (^	K L + M N* - O P
)	+	K L + M N* - O P ^

Infix expression: K + L - M*N + (O^P) * W/U/V * T + Q

*	+ *	K L + M N* - O P ^
W	+ *	K L + M N* - O P ^ W
/	+ /	K L + M N* - O P ^ W *
U	+ /	K L + M N* - O P ^W*U
/	+ /	K L + M N* - O P ^W*U/
V	+ /	KL + MN*-OP^W*U/V
*	+ *	KL+MN*-OP^W*U/V/
T	+ *	KL+MN*-OP^W*U/V/T
+	+	KL+MN*-OP^W*U/V/T* KL+MN*-OP^W*U/V/T*+
Q	+	KL+MN*-OP^W*U/V/T*Q
		KL+MN*-OP^W*U/V/T*+Q+

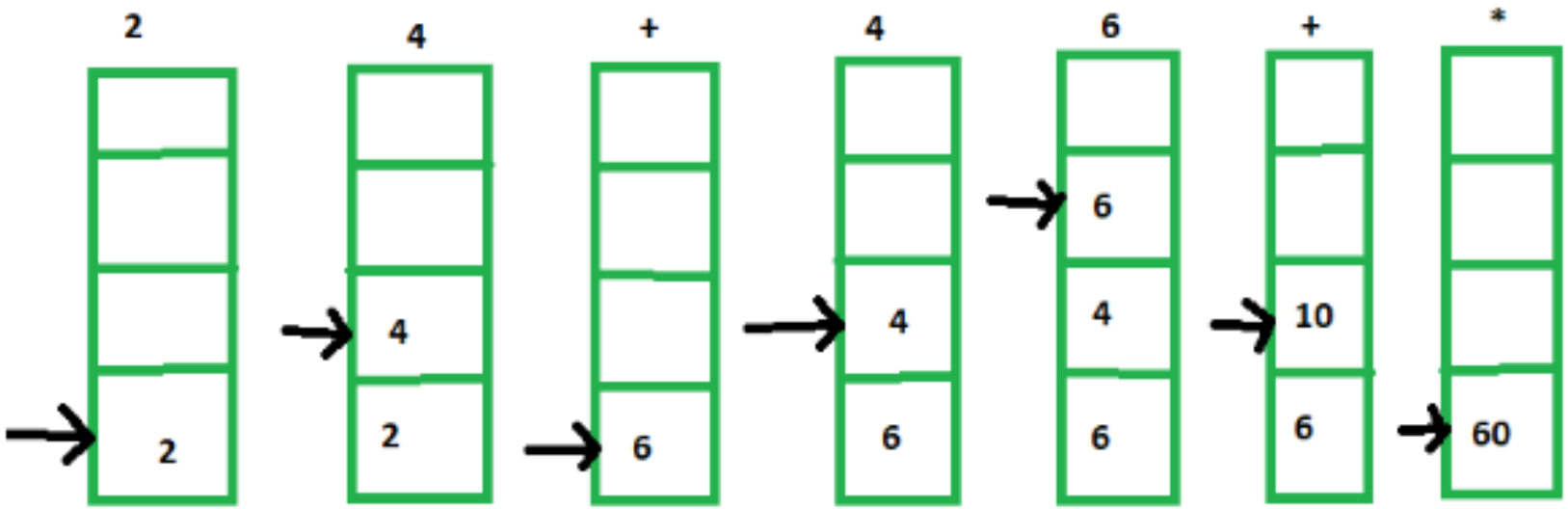
The final postfix expression of infix expression(K + L - M*N + (O^P) * W/U/V * T + Q) is KL+MN*-OP^W*U/V/T*+Q+.

```
Infix notation: (2+4) * (4+6)
Post-fix notation: 2 4 + 4 6 + *
Result: 60
```

24+ * 46+

24+46+*

The stack operations for this expression evaluation is shown below:



1) 7 5 2 + * 4 1 5 − / −

$\boxed{\begin{matrix}2\\5\\7\end{matrix}}$ + $\boxed{\begin{matrix}7\\7\end{matrix}}$ 5+2=7 * $\boxed{\begin{matrix}5\\1\\4\\49\end{matrix}}$ 7*7=49 − $\boxed{\begin{matrix}-4\\4\\49\end{matrix}}$ 1−5 = −4

/ $\boxed{\begin{matrix}-4\\4\\49\end{matrix}}$ 4/−4 = −1 $\boxed{\begin{matrix}-1\\49\end{matrix}}$ − $\boxed{50}$ 49 − (−1) = 49+1 = 50

50/

2) 562 + * 84 / -

2
6
5

+

8
5

6+2=8

*

4
8
40

5*8=40

/

2
40

8/4=2

-

38

40-2=38

Infix to Postfix conversion

(A – B) * (D / E)

(A + B ↑ D) / (E – F) + G

A * (B + D) / E – F * (G + H / K)

Practice:

Write an algorithm to print all the elements of the stack.

Write an algorithm to check whether an element is present in the stack or not

Algorithm

Evaluation of Postfix expression

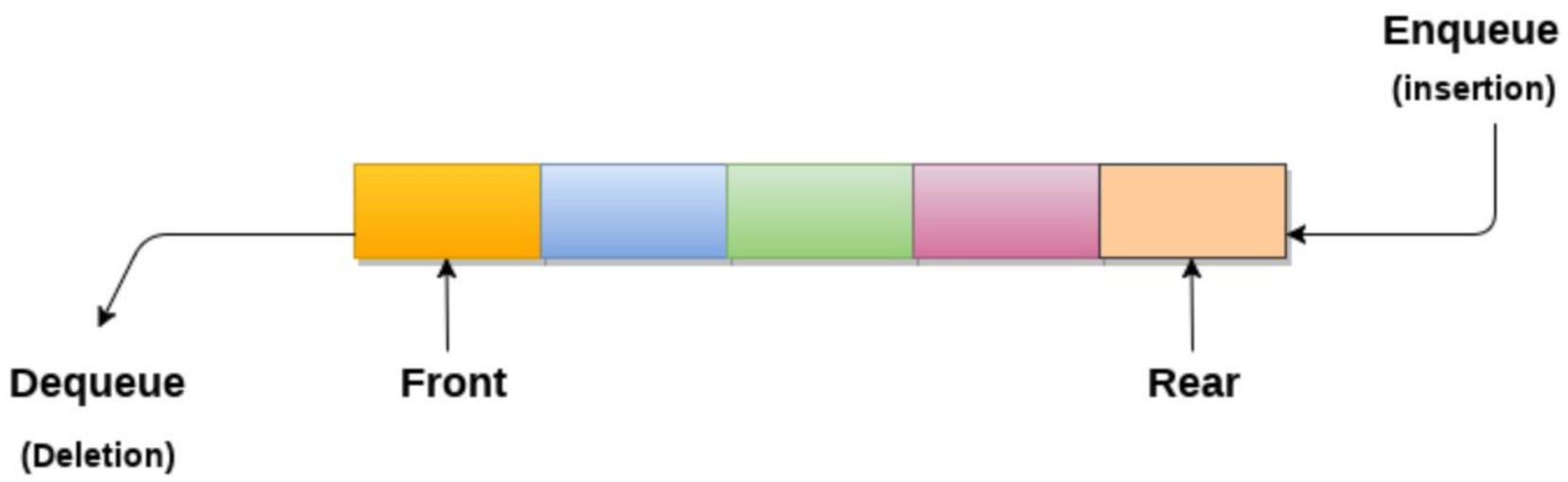
Conversion from infix to postfix

Algorithm

- Evaluation of postfix expression
- Scan the input string from left to right.
- For each input symbol,
- If it is a digit then, push it on to the stack.
- If it is an operator then, pop out the top most two contents from the stack and apply the operator on them. Later on, push the result on to stack.
- If the input symbol is ‘\0’, empty the stack.

Simple Queue

- A queue can be defined as an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.
- Queue is referred to be as First In First Out list.
- For example, people waiting in line for a rail ticket form a queue.



Queue - Applications

1. Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.
2. Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.
3. Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.
4. Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.
5. Queues are used in operating systems for handling interrupts.

Queue - Applications

As we now understand that in queue, we access both ends for different reasons. The following diagram given below tries to explain queue representation as data structure –

As in stacks, a queue can also be implemented using Arrays, Linked-lists, Pointers and Structures.

Queue - operations

- Queue operations may involve initializing or defining the queue, utilizing it, and then completely erasing it from the memory. Here we shall try to understand the basic operations associated with queues –
- **enqueue()** – add (store) an item to the queue.
- **dequeue()** – remove (access) an item from the queue.
- Few more functions are required to make the above-mentioned queue operation efficient. These are –
- **peek()** – Gets the element at the front of the queue without removing it.
- **isfull()** – Checks if the queue is full.
- **isempty()** – Checks if the queue is empty.
- In queue, we always dequeue (or access) data, pointed by **front** pointer and while enqueing (or storing) data in the queue we take help of **rear** pointer.

Queue - operations

enQueue(value) - Inserting value into the queue

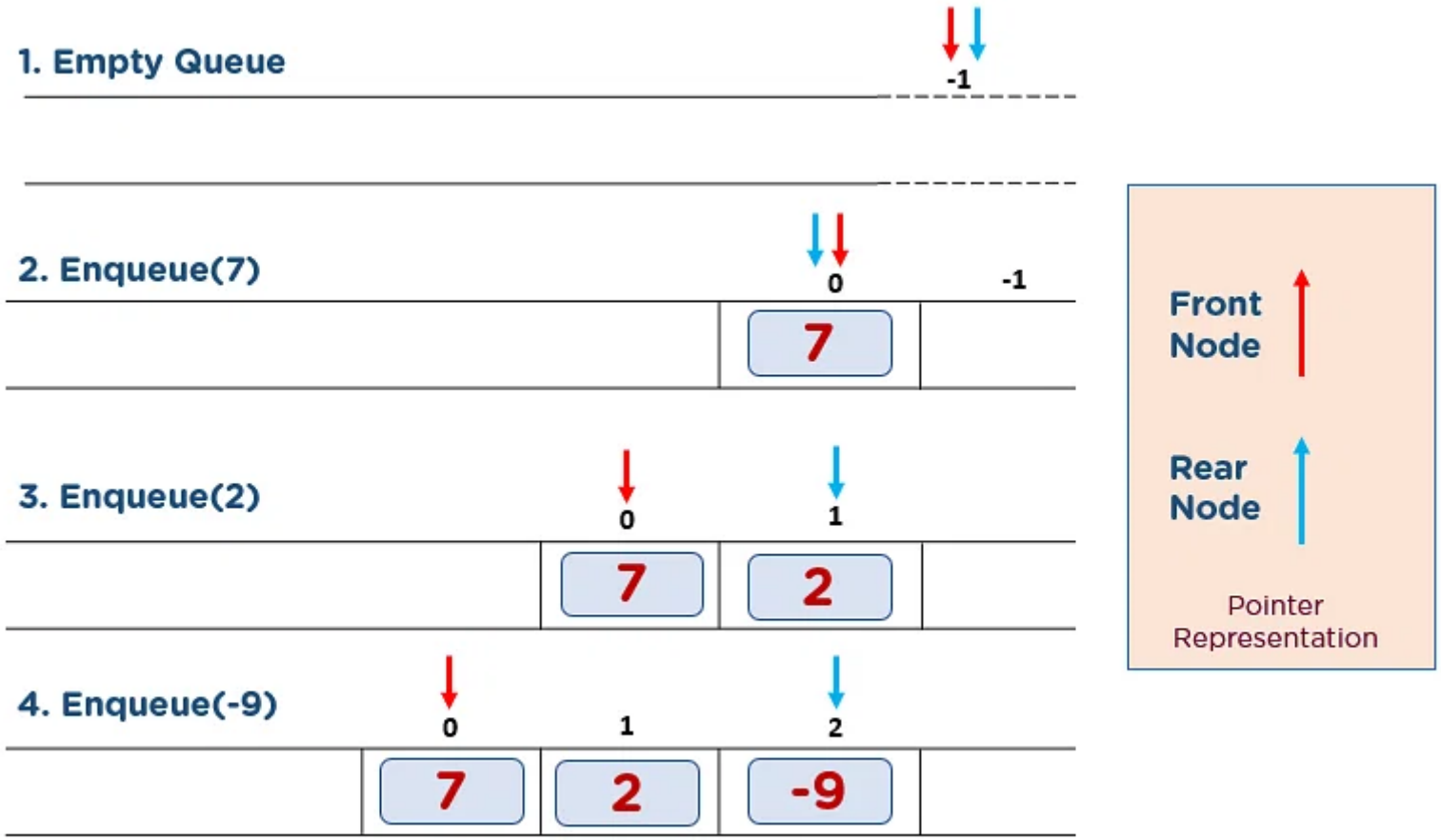
Step 1: Check if the queue is full.

Step 2: If the queue is full, Overflow error.

Step 3: If the queue is not full, increment the rear pointer to point to the next available empty space.

Step 4: Add the data element to the queue location where the rear is pointing.

Step 5: Here, you have successfully added 7, 2, and -9.

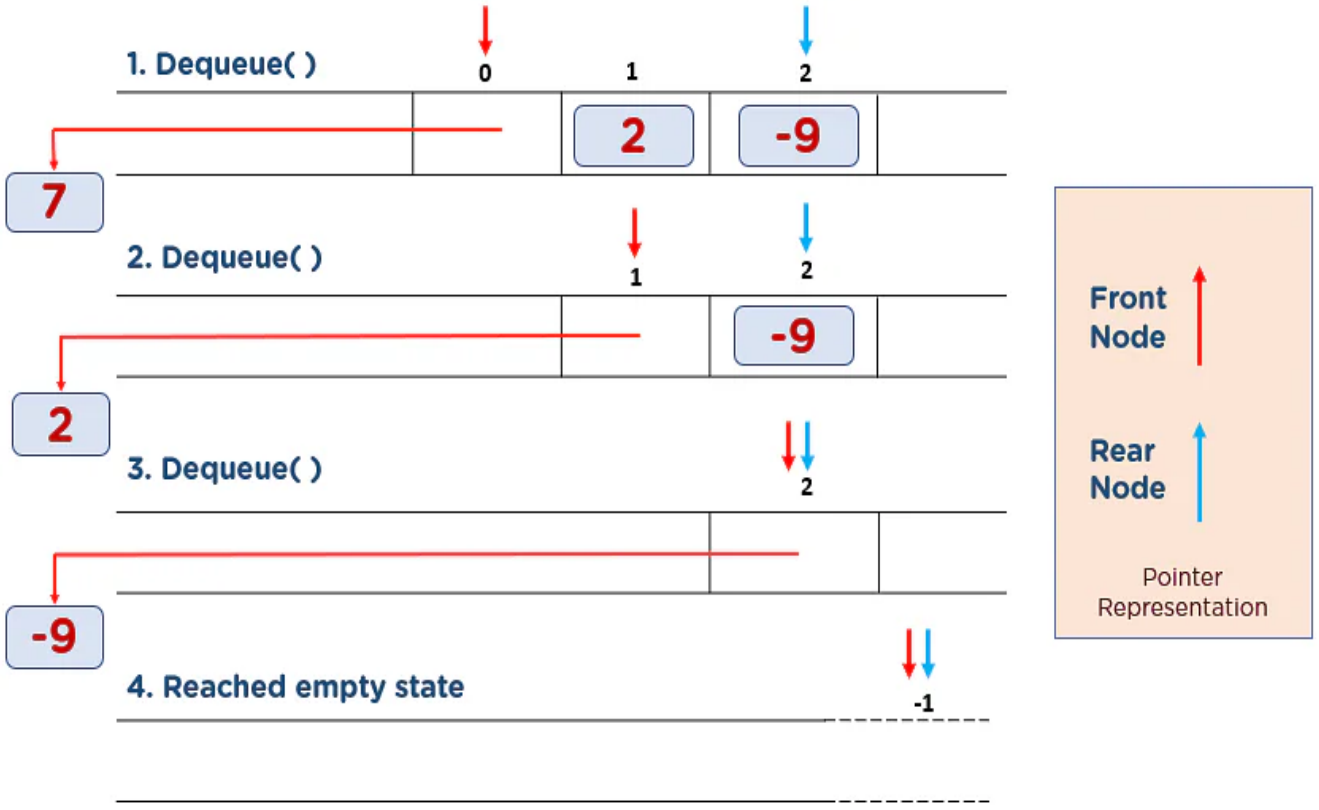


Queue - operations

deQueue() - Deleting a value from the Queue

In a queue data structure, deQueue() is a function used to delete an element from the queue.
In a queue, the element is always deleted from **front** position.
The deQueue() function does not take any value as parameter.
We can use the following steps to delete an element from the queue...

Step 1: Check if the queue is empty.
Step 2: If the queue is empty, Underflow error.
Step 3: If the queue is not empty, access the data where the front pointer is pointing.
Step 4: Increment front pointer to point to the next available data element.
Step 5: Here, you have removed 7, 2, and -9 from the queue data structure.



Queue - operations

display() - Displays the elements of a Queue

We can use the following steps to display the elements of a queue...

•**Step 1 -** Check whether **queue** is **EMPTY**.

•**Step 2 -** If it is **EMPTY**, then display **"Queue is EMPTY!!!"** and terminate the function.

•**Step 3 -** If it is **NOT EMPTY**, then define an integer variable '**i**' and set '**i** = **front+1**'.

•**Step 4 -** Display '**queue[i]**' value and increment '**i**' value by one (**i++**). Repeat the same until '**i**' value reaches to **rear** (**i** <= **rear**)

Queue - operations

Peek() Operation

This function helps in extracting the data element where the front is pointing without removing it from the queue. The algorithm of Peek() function is as follows-

Step 1: Check if the queue is empty.
Step 2: If the queue is empty, return “Queue is Empty.”
Step 3: If the queue is not empty, access the data where the front pointer is pointing.
Step 4: Return data.

isFull() Operation

This function checks if the rear pointer is reached at MAXSIZE to determine that the queue is full. The following steps are performed in the isFull() operation –

Step 1: Check if rear == MAXSIZE - 1.
Step 2: If they are equal, return “Queue is Full.”
Step 3: If they are not equal, return “Queue is not Full.”

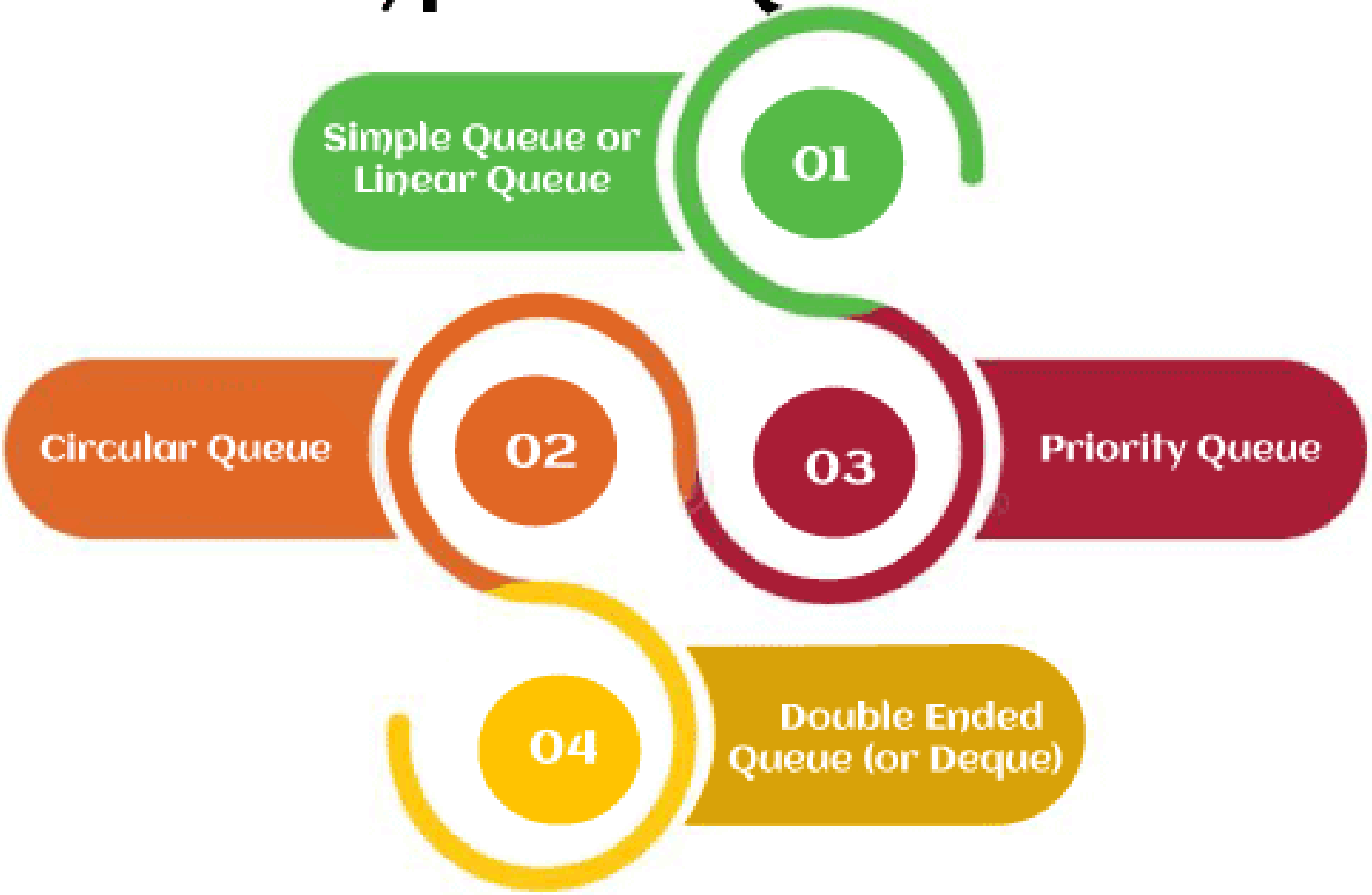
Queue - operations

isEmpty() Operation

The algorithm of the isEmpty() operation is as follows –

Step 1: Check if the rear and front are pointing to null memory space, i.e., -1. // front=-1
Step 2: If they are pointing to -1, return “Queue is empty.”
Step 3: If they are not equal, return “Queue is not empty.”

Types of Queues

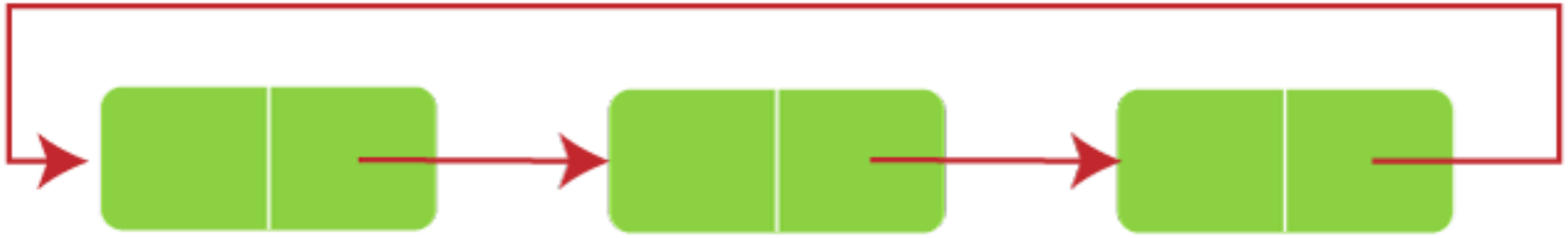


Circular Queue

In Circular Queue, all the nodes are represented as circular.

It is similar to the linear Queue except that the last element of the queue is connected to the first element.

It is also known as Ring Buffer, as all the ends are connected to another end. The representation of circular queue is shown in the below image –



Circular Queue

The drawback that occurs in a linear queue is overcome by using the circular queue.

If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear.

The main advantage of using the circular queue is better memory utilization.

Circular Queue - Insert

- **Algorithm to insert an element in a circular queue**
- **Step 1:** IF (REAR+1)%MAX = FRONT
  Write " OVERFLOW "
  Goto step 4
  [End OF IF]
- **Step 2:** IF FRONT = -1 and REAR = -1
  SET FRONT = REAR = 0

ELSE
SET REAR = (REAR + 1) % MAX
[END OF IF]

- **Step 3:** SET QUEUE[REAR] = VAL
- **Step 4:** EXIT

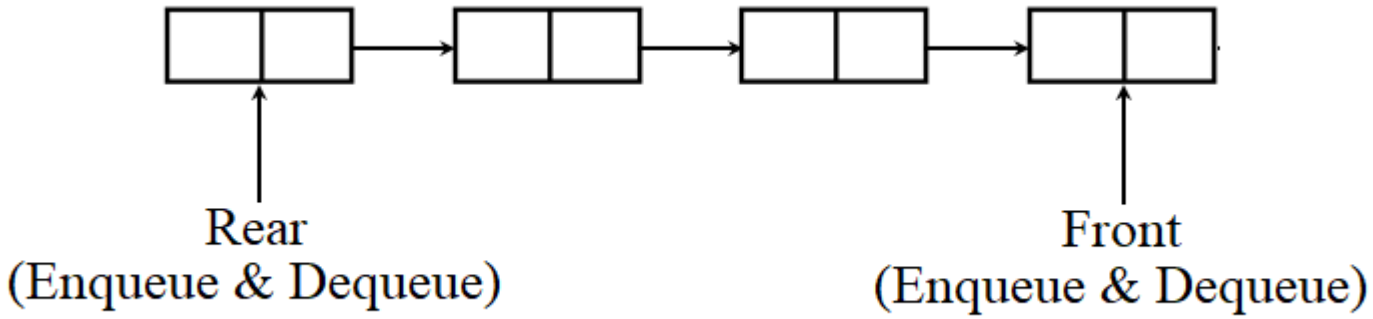
Circular Queue – Delete

- **Step 1:** IF FRONT = -1
  Write " UNDERFLOW "
  Goto Step 4
  [END of IF]
- **Step 2:** SET VAL = QUEUE[FRONT]
- **Step 3:** IF FRONT = REAR
  SET FRONT = REAR = -1
  ELSE
  SET FRONT = (FRONT + 1)%N

  [END OF IF]
- **Step 4:** EXIT

Deque (or, Double Ended Queue)

- A deque is also a special type of queue.
- In this queue, the **enqueue and dequeue operations take place at both front and rear**.
- That means, we can insert an item at both the ends and can remove an item from both the ends. Thus, it may or may not adhere to the FIFO order:

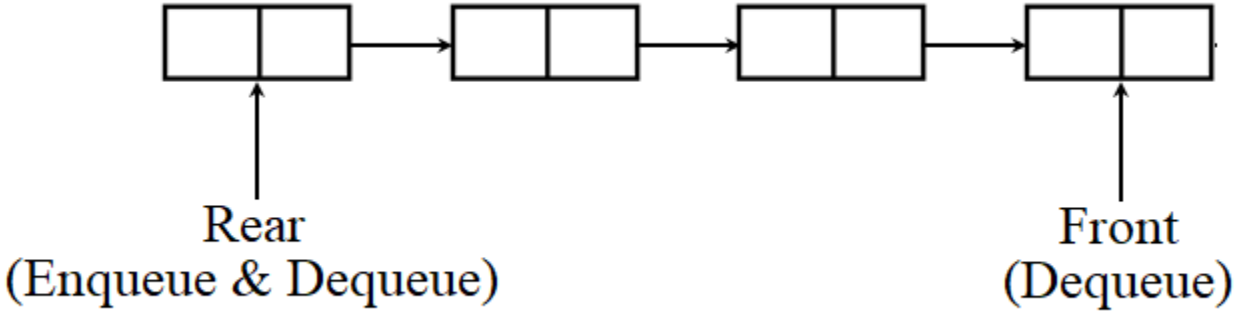


Deque (or, Double Ended Queue)

- **<u>Properties of Deque:</u>**
- Deque is a generalized version of the queue which allows us to insert and delete the element at both ends.
- It does not follow FIFO (first in first out) rule.
- **Applications:**
- Applied as both stack and queue, as it supports both operations.
- Storing a web browser's history: In a web browser's history, recently visited URLs are added to the front of the deque and the URL at the back of the deque is removed after some specified number of operations of insertions at the front.
- Storing a software application's list of undo operations.
- Job scheduling algorithm

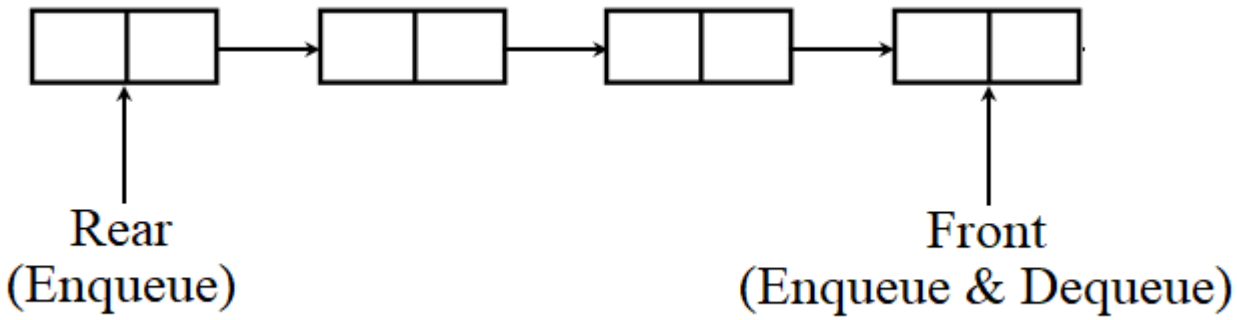
Deque (or, Double Ended Queue)

- Further, it has two special cases: **input-restricted deque** and **output-restricted deque**.
- As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.
- The enqueue operation takes place only at the rear, but the dequeue operation takes place at both rear and front:
-



Deque (or, Double Ended Queue)

- As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.
- The enqueue operation takes place at both rear and front, but the dequeue operation takes place only at the front:



- An output-restricted queue is handy when we need to insert an item at the front of the queue.

Deque (or, Double Ended Queue)

- **Operations on Deque:** Mainly the following four basic operations are performed on queue:

insertFront(): Adds an item at the front of Deque.
insertLast(): Adds an item at the rear of Deque.
deleteFront(): Deletes an item from the front of Deque.
deleteLast(): Deletes an item from the rear of Deque. In addition to the above operations, the following operations are also supported.
getFront(): Gets the front item from the queue.
getRear(): Gets the last item from queue.

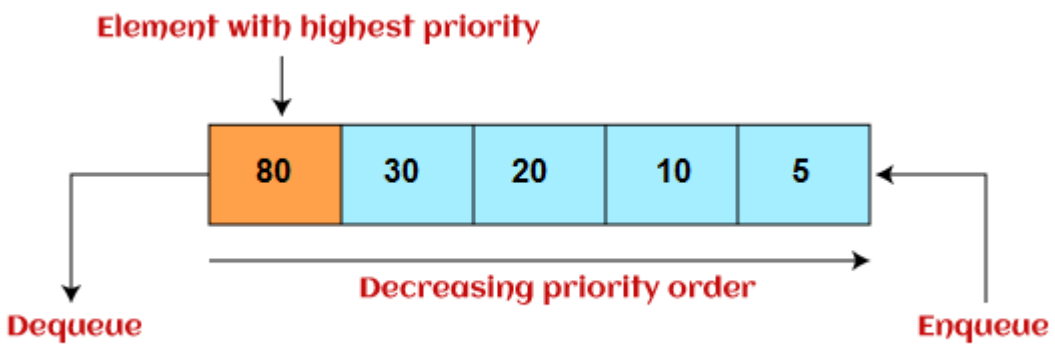
Priority Queue

It is a special type of queue in which the elements are arranged based on the priority.

It is a special type of queue data structure in which every element has a priority associated with it.

Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle.

The representation of priority queue is shown in the below image -

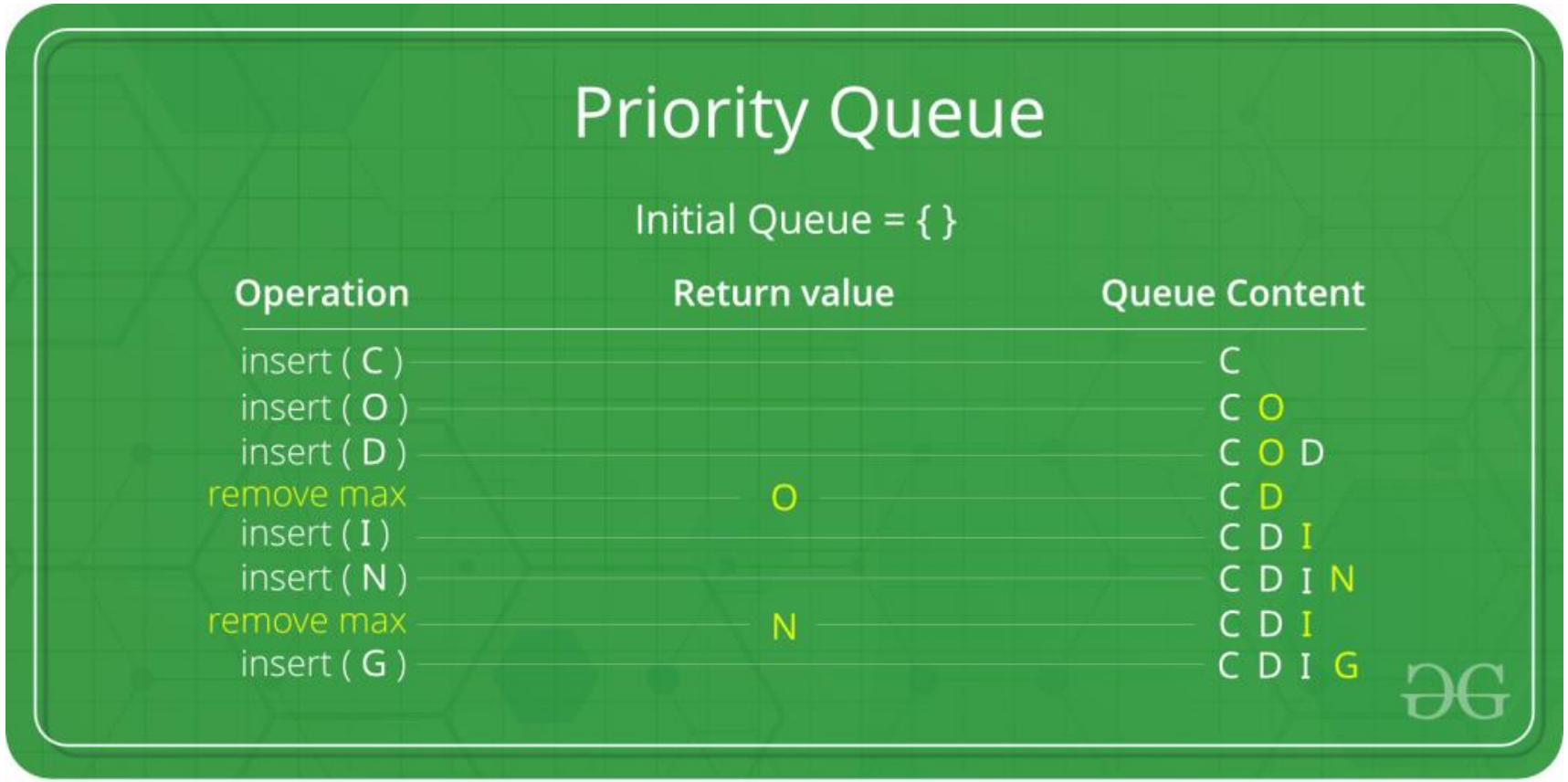


Priority Queue

- Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.
- There are two types of priority queue that are discussed as follows -
- **Ascending priority queue -** In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.
- **Descending priority queue -** In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

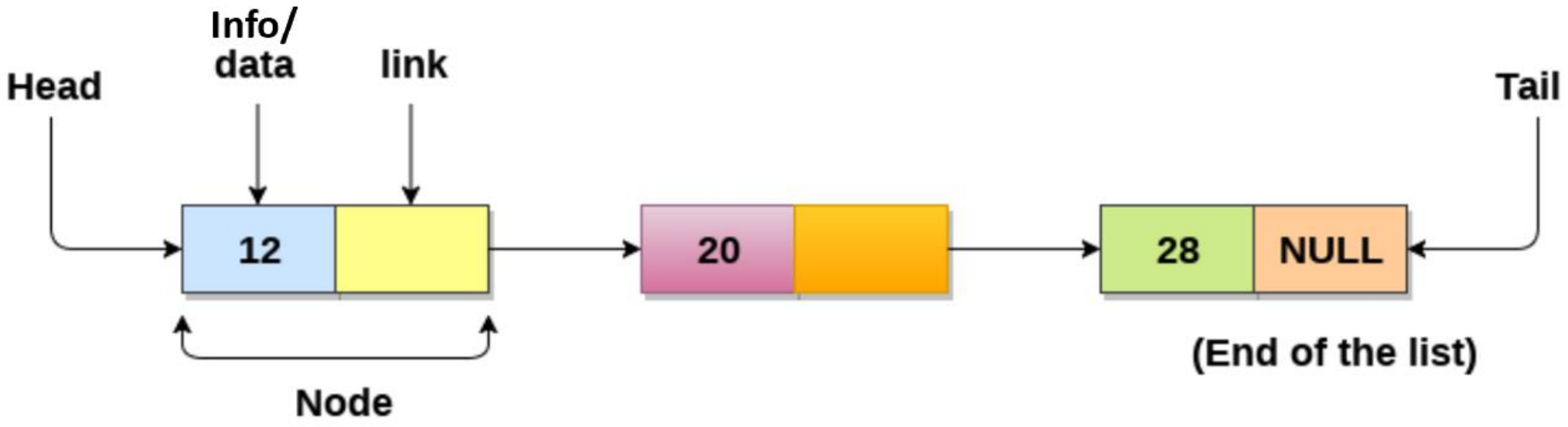
Priority Queue

- In the below priority queue, an element with a maximum ASCII value will have the highest priority. The elements with higher priority are served first.



Linked List

- Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.



Linked List

- The list is not required to be contiguously present in the memory. The node can reside any where in the memory and linked together to make a list. This achieves optimized utilization of space.
- list size is limited to the memory size and doesn't need to be declared in advance.
- We can store values of primitive types or objects in the singly linked list.
- **Note:**
- linked list required extra space for storing pointer variable
- Sequential traversal is only possible. We can not access any node from middle. If we want to access last node then we have to start traversal from first node. So time complexity to access any element is O(n) in worst case because we need to visit every node. For accessing first node time complexity is O(1).
- For array to access any element time complexity is O(1).
- Insertion and deletion is easier than array

Why to use Linked List over array

Array contains following limitations:

1. The size of array must be known in advance before using it in the program.
2. Increasing size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.
3. All the elements in the array need to be contiguously stored in the memory. Inserting any element in the array needs shifting of all its predecessors.

Linked list is the data structure which can overcome all the limitations of an array. Using linked list is useful because,

1. It allocates the memory dynamically. All the nodes of linked list are non-contiguously stored in the memory and linked together with the help of pointers.
2. Sizing is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

Linked List – Basic Operations

- Traversal - access each element of the linked list
- Insertion - adds a new element to the linked list
- Deletion - removes the existing elements
- Search - find a node in the linked list
- Sort - sort the nodes of the linked list

Types of Linked List

Following are the various types of linked list.

- **Simple Linked List** − Item navigation is forward only.
- **Doubly Linked List** − Items can be navigated forward and backward.
- **Circular Linked List** − Last item contains link of the first element as next and the first element has a link to the last element as previous.

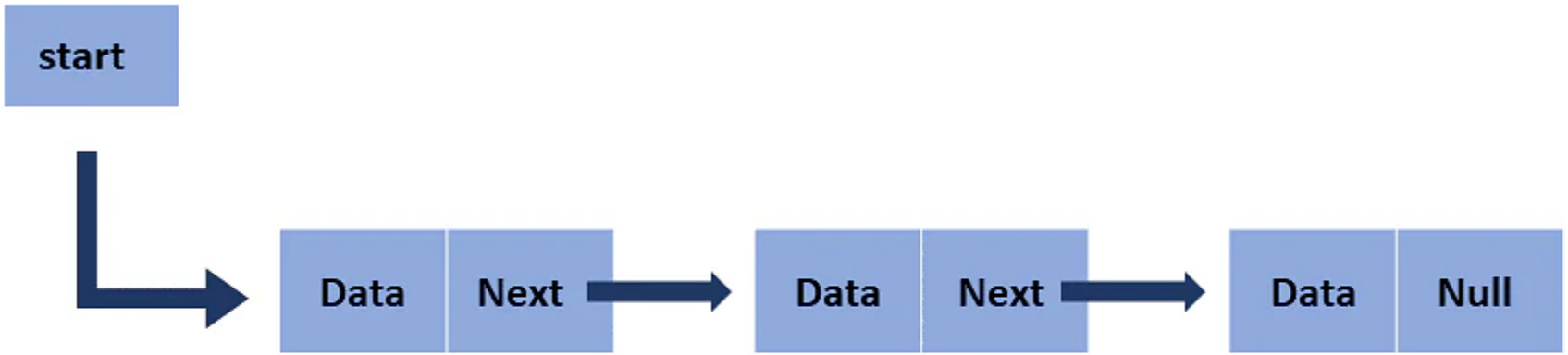
Singly Linked List

- Singly linked list can be defined as the collection of ordered set of elements.
- The number of elements may vary according to need of the program.
- A node in the singly linked list consist of two parts: info part and link part.
- Info part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.
- One way chain or singly linked list can be traversed only in one direction.
- In other words, we can say that each node contains only next pointer, therefore we can not traverse the list in the reverse direction.

Singly Linked List – Important points

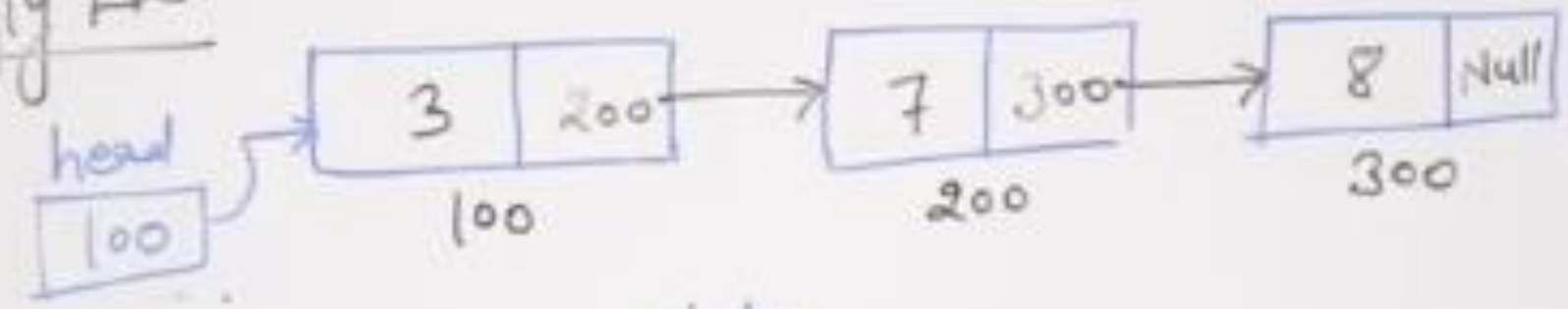
- Start or head points to the first node of the linked list
- Link contains the address of the next node.
- link pointer of the last node is NULL.

Singly Linked List – Memory representation

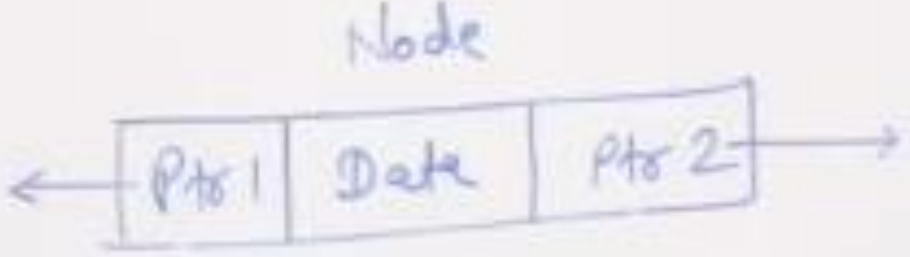


Types of Linked list

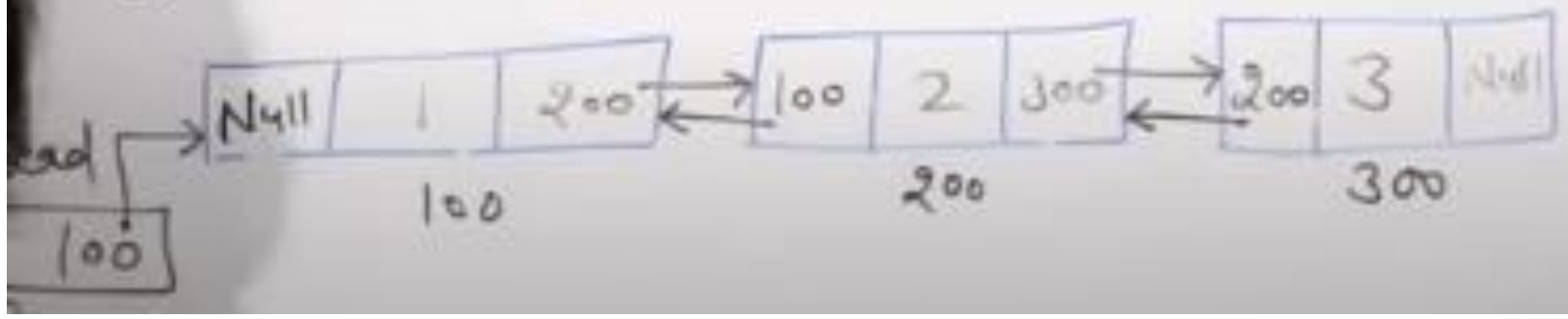
Singly LL :-




```
struct node
{ int data;
struct node * next;
};
```

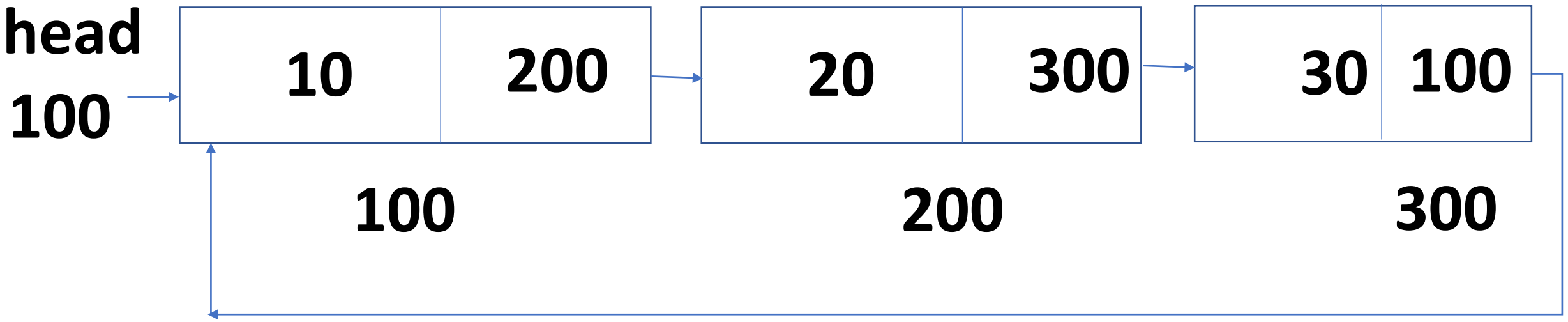


Doubly LL :-




```
struct node
{
    int data,
struct node * next;
struct node * prev;
};
```

- Circular linked list



Singly Linked List – Node creation - How to declare a linked list?

- It is simple to declare an array, as it is of single type, while the declaration of linked list is a bit more typical than array. Linked list contains two parts, and both are of different types, i.e., one is the simple variable, while another is the pointer variable. We can declare the linked list by using the user-defined data type **structure.**
- The declaration of linked list is given as follows -

```
struct node
{
  int info;
  struct node *next;
};
```

In the above declaration, we have defined a structure named as **node** that contains two variables, one is **data** that is of integer type, and another one is **next** that is a pointer which contains the address of next node.

Singly Linked List – Node creation

Node creation and declaration

```
struct node
{
 int data;
 struct node * next;
};
struct node * n;
n=(struct node*)malloc(sizeof(struct node*));
```

- It is a declaration of a node that consists of the first variable as data and the next as a pointer, which will keep the address of the next node.
- Here you need to use the malloc function to allocate memory for the nodes dynamically.

Singly Linked List – Insert

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

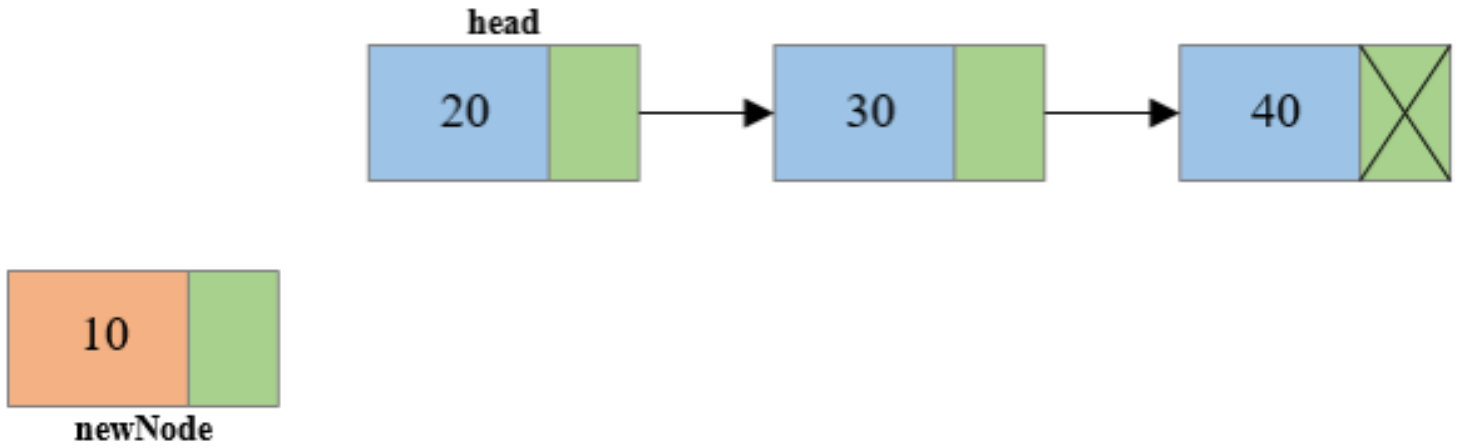
SN	Operation	Description
1	Insertion at beginning	It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.
2	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario.
3	Insertion after specified node	It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted. .

Algorithm to insert node at the beginning of singly linked list

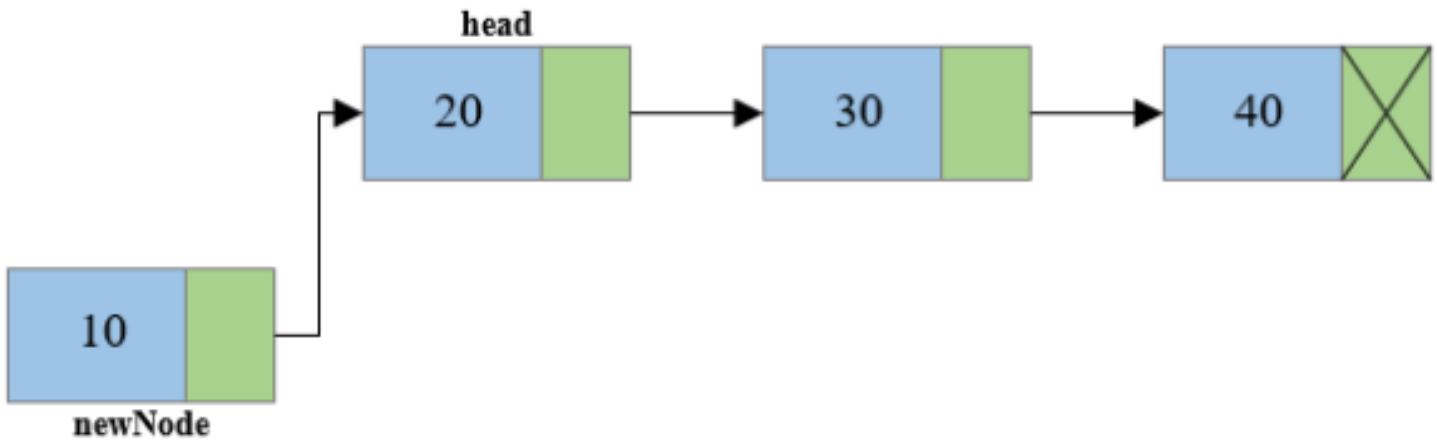
```
Algorithm to insert node at the beginning of Singly Linked List
Being:
    createSinglyLinkedList (head)
    alloc (newNode)
    If (newNode == NULL) then
        write ('Unable to allocate memory')
    End if
    Else then
        read (data)wo
        newNode.data ← data
        newNode.next ← head
        head ← newNode
    End else
End
```

Steps to insert node at the beginning of singly linked list

1. Create a new node, say `newNode` points to the newly created node.

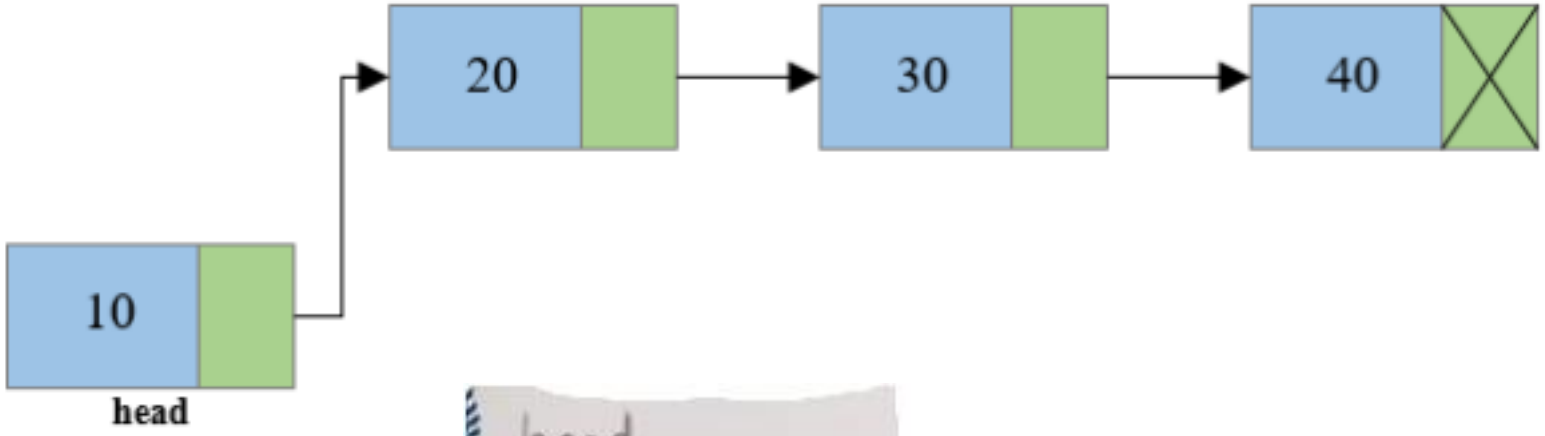


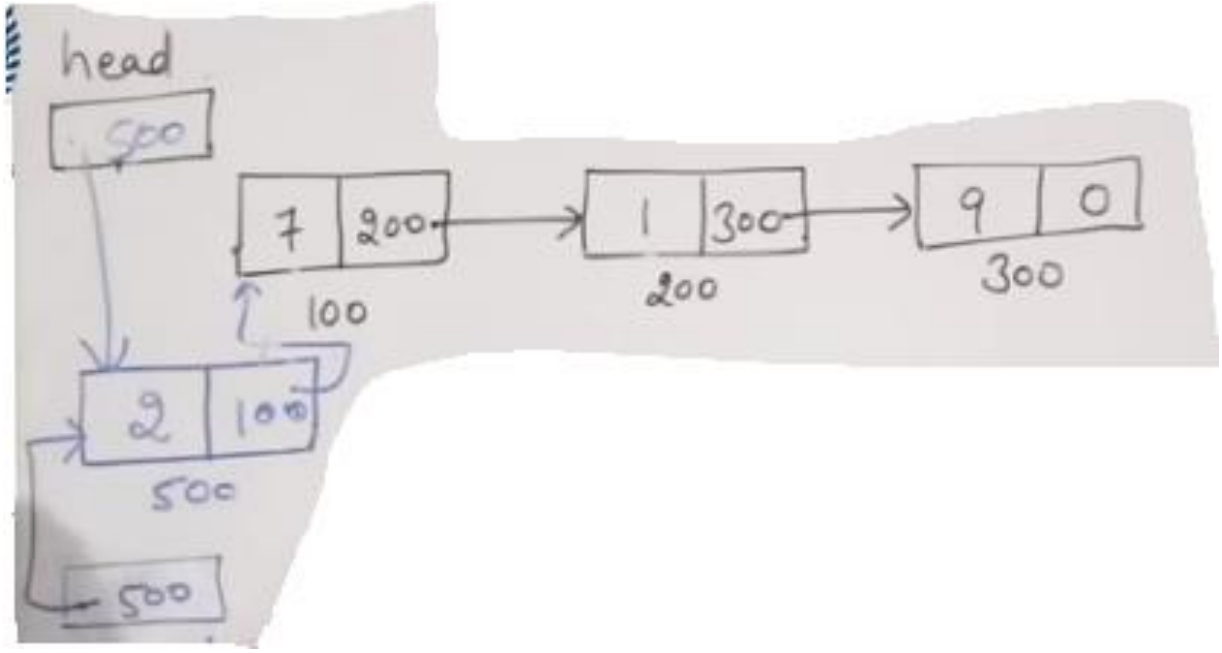
2. Link the newly created node with the head node, i.e. the `newNode` will now point to `head` node.



Steps to insert node at the beginning of singly linked list

3. Make the new node as the head node, i.e. now `head` node will point to `newNode` .

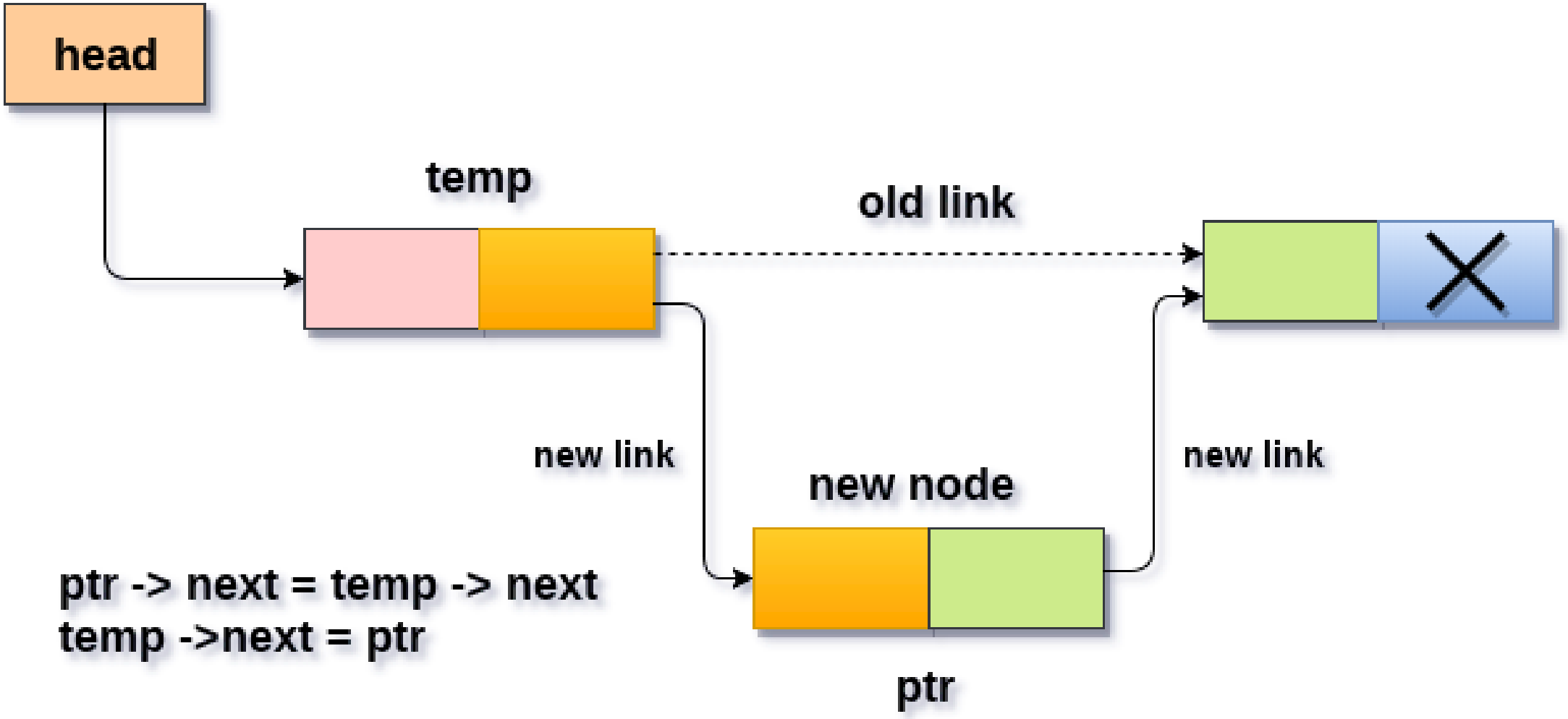




Algorithm to insert node at the end of Singly linked list

```
Algorithm to insert node at the end of a Singly Linked List
Begin:
    createSinglyLinkedList (head)
    alloc (newNode)
    If (newNode == NULL) then
        write ('Unable to allocate memory')
    End if
    Else then
        read (data)
        newNode.data ← data
        newNode.next ← NULL
        temp ← head
        While (temp.next != NULL) do
            temp ← temp.next
        End while
        temp.next ← newNode
    End else
End
```

Singly Linked List – Insert at specified node



ptr -> next = temp -> next
temp ->next = ptr

Singly Linked List – deletion and traversing

The Deletion of a node from a singly linked list can be performed at different positions. Based on the position of the node being deleted, the operation is categorized into the following categories.

SN	Operation	Description
1	Deletion at beginning	It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.
2	Deletion at the end of the list	It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios.
3	Deletion after specified node	It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list.
4	Traversing	In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list.
5	Searching	In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned. .

Singly Linked List – Deletion in singly linked list at beginning

Deleting a node from the beginning of the list is the simplest operation of all.
It just need a few adjustments in the node pointers.
Since the first node of the list is to be deleted, therefore, we just need to make the head, point to the next of the head.
This will be done by using the following statements.

```
ptr = head;
        head = ptr->next;
```

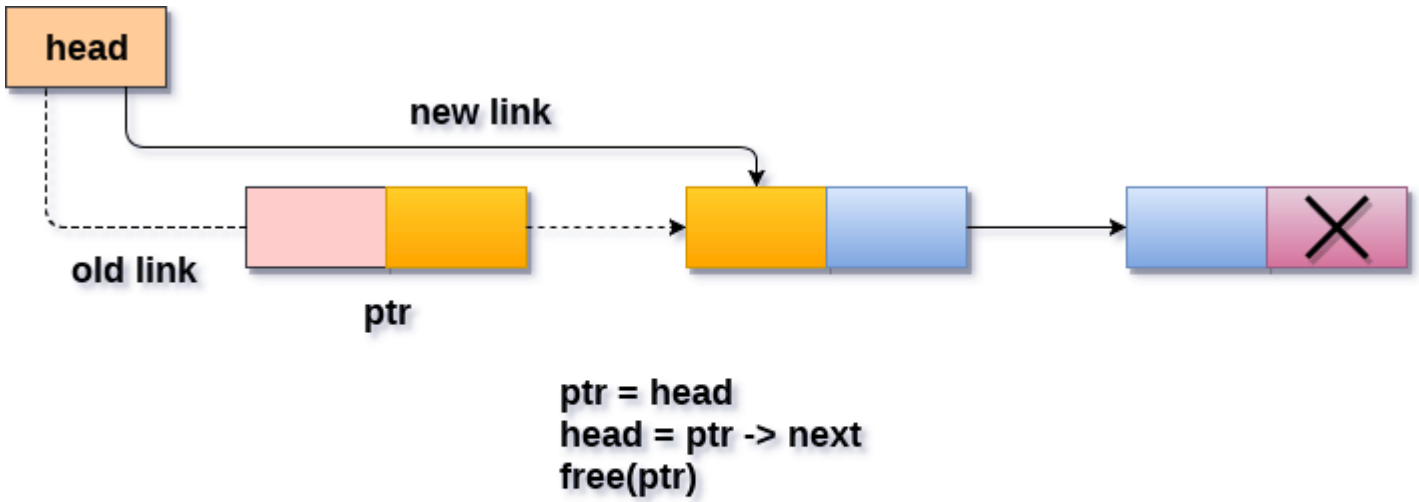
Now, free the pointer ptr which was pointing to the head node of the list. This will be done by using the following statement.

```
free(ptr)
```

Singly Linked List – Deletion in singly linked list at beginning

Algorithm

Step 1: IF HEAD = NULL
Write UNDERFLOW
    Go to Step 5
    [END OF IF]
Step 2: SET PTR = HEAD
Step 3: SET HEAD = HEAD -> NEXT
Step 4: FREE PTR
Step 5: EXIT



Deleting a node from the beginning

Singly Linked List – Deletion in singly linked list at end

There are two scenarios in which, a node is deleted from the end of the linked list.
There is only one node in the list and that needs to be deleted.
There are more than one node in the list and the last node of the list will be deleted.
In the first scenario,
the condition head → next = NULL will survive and therefore, the only node head of the list will be assigned to null.
This will be done by using the following statements.

```
ptr = head
    head = NULL
    free(ptr)
```

In the second scenario,
The condition head → next = NULL would fail and therefore, we have to traverse the node in order to reach the last node of the list.
For this purpose, just declare a temporary pointer temp and assign it to head of the list. We also need to keep track of the second last node of the list. For this purpose, two pointers ptr and ptr1 will be used where ptr will point to the last node and ptr1 will point to the second last node of the list.

Singly Linked List – Deletion in singly linked list at end

this all will be done by using the following statements.

```
ptr = head;
        while(ptr->next != NULL)
        {
            ptr1 = ptr;
            ptr = ptr ->next;
        }
```

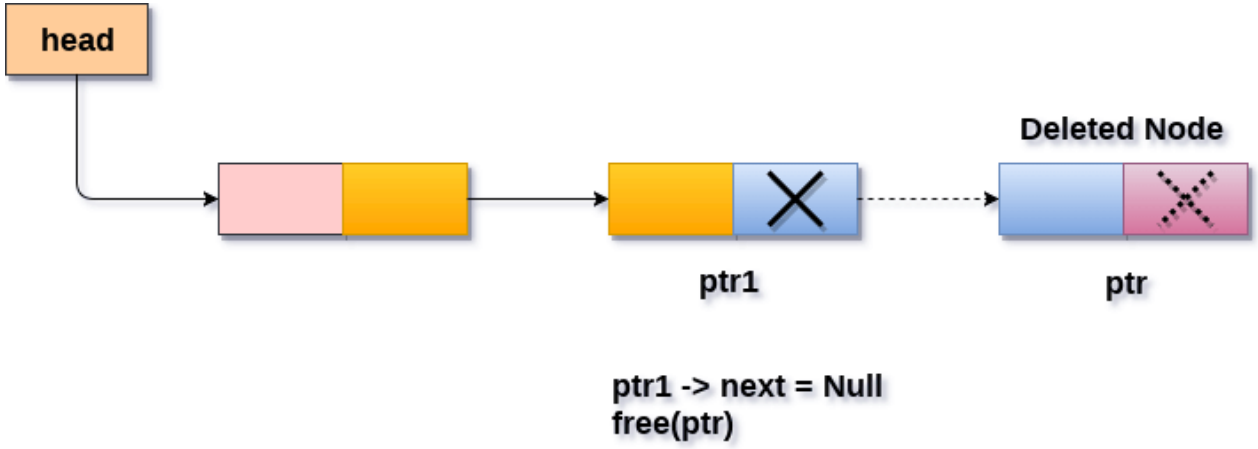
Now, we just need to make the pointer ptr1 point to the NULL and the last node of the list that is pointed by ptr will become free. It will be done by using the following statements.

```
ptr1->next = NULL;
        free(ptr);
```

Singly Linked List – Deletion in singly linked list at end

Algorithm

Step 1: IF HEAD = NULL
Write UNDERFLOW
  Go to Step 8
 [END OF IF]
Step 2: SET PTR = HEAD
Step 3: Repeat Steps 4 and 5 while PTR -> NEXT!= NULL
Step 4: SET PREPTR = PTR
Step 5: SET PTR = PTR -> NEXT
[END OF LOOP]
Step 6: SET PREPTR -> NEXT = NULL
Step 7: FREE PTR
Step 8: EXIT



Deleting a node from the last

Singly Linked List – Deletion in singly linked list at specified position

In order to delete the node, which is present after the specified node, we need to skip the desired number of nodes to reach the node after which the node will be deleted. We need to keep track of the two nodes. The one which is to be deleted the other one if the node which is present before that node. For this purpose, two pointers are used: ptr and ptr1.
Use the following statements to do so.

```
ptr=head;
    for(i=0;i<loc;i++)
    {
        ptr1 = ptr;
        ptr = ptr->next;

        if(ptr == NULL)
        {
            printf("\nThere are less than %d elements in the list..",loc);
            return;
        }
    }
```

Singly Linked List – Deletion in singly linked list at specified position

Now, our task is almost done, we just need to make a few pointer adjustments. Make the next of ptr1 (points to the specified node) point to the next of ptr (the node which is to be deleted).
This will be done by using the following statements.

```
ptr1 ->next = ptr ->next;
      free(ptr);
```

Singly Linked List – Deletion in singly linked list at specified position

Algorithm

STEP 1: IF HEAD = NULL
WRITE UNDERFLOW
GOTO STEP 10
END OF IF

STEP 2: SET TEMP = HEAD

STEP 3: SET I = 0

STEP 4: REPEAT STEP 5 TO 8 UNTIL I

STEP 5: TEMP1 = TEMP

STEP 6: TEMP = TEMP → NEXT

STEP 7: IF TEMP = NULL
WRITE "DESIRED NODE NOT PRESENT"
GOTO STEP 12
END OF IF

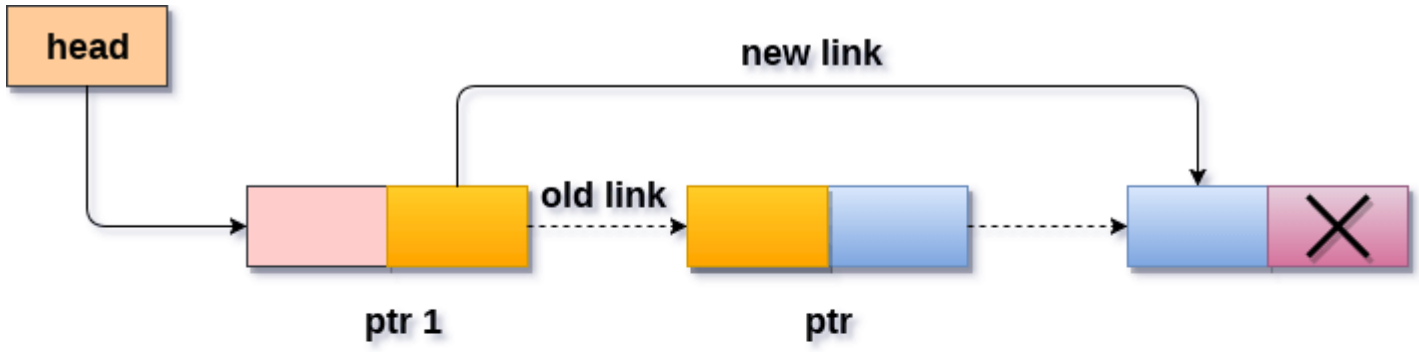
STEP 8: I = I+1
END OF LOOP

STEP 9: TEMP1 → NEXT = TEMP → NEXT

STEP 10: FREE TEMP

STEP 11: EXIT

Singly Linked List – Deletion in singly linked list at specified position



ptr1 -> next = ptr -> next
free(ptr)

Deletion a node from specified position

Singly Linked List – Traversing in singly linked list

Traversing is the most common operation that is performed in almost every scenario of singly linked list. Traversing means visiting each node of the list once in order to perform some operation on that. This will be done by using the following statements.

```
ptr = head;
      while (ptr!=NULL)
         {
            ptr = ptr -> next;
         }
```

Algorithm

STEP 1: SET PTR = HEAD

STEP 2: IF PTR = NULL

WRITE "EMPTY LIST"

GOTO STEP 7

END OF IF

STEP 4: REPEAT STEP 5 AND 6 UNTIL PTR != NULL

STEP 5: PRINT PTR→ DATA

STEP 6: PTR = PTR → NEXT

[END OF LOOP]

STEP 7: EXIT

Singly Linked List – search in singly linked list

searchNode() will search for a node in the list:

Variable i will keep track of the position of the searched node.
The variable flag will store boolean value false.
Node temp will point to head node.
Iterate through the loop by incrementing temp to temp.next and i to i + 1.
Compare each node's data with the searched node. If a match is found, set flag to true.
If the flag is true, display the position of the searched node.
Else, display the message "Element is not present in the list".

Singly Linked List – sort singly linked list

sortList() will sort the nodes of the list in ascending order.

Define a node current which will point to head.

Define another node index which will point to node next to current.

Compare data of current and index node. If current's data is greater than the index's data then, swap the data between them.

Current will point to current.next and index will point to index.next.

Continue this process until the entire list is sorted.

Singly Linked List – union and intersection singly linked list

Given two singly linked list, we have to create two linked list which contains **union and intersection of both given linked list** respectively. For Example:

```
First Linked List
1-->2-->4-->9-->6-->8
Second Linked List
3-->1-->6-->10

Union of Both Linked List
1-->2-->4-->9-->6-->8-->3-->10
Intersection of Both Linked List
1-->6
```

Singly Linked List – union and intersection singly linked list

Algorithm to find union of two linked list

Let the two input linked list be LLOne and LLTwo and unionLL be the result linked list.
Initialize unionLL with NULL.
Traverse LLone and add all nodes to unionLL.
Traverse LLTwo and check if a node of LLtwo is already present in unionLL then don't add this node otherwise add it in unionLL.

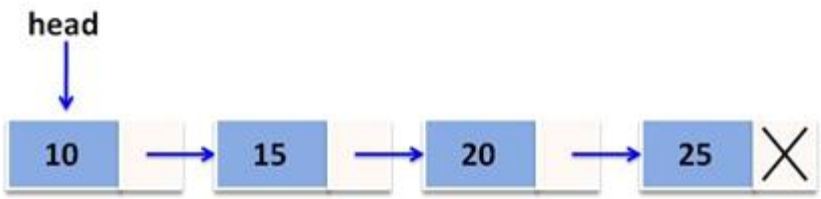
Algorithm to find intersection of two linked list

Let the two input linked list be LLOne and LLTwo and intersectionLL be the result linked list.
Initialize intersectionLL with NULL.
Traverse LLOne, search each node of LLOne in LLTwo linked list.
If found then add it to intersectionLL otherwise continue.

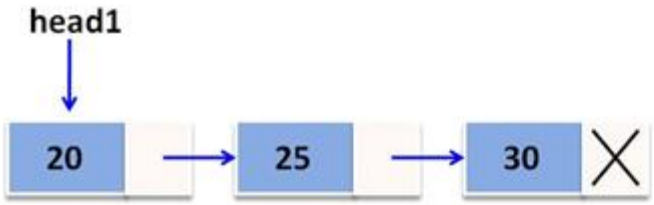
Union Operation on linked list

- **Union is performed on 2 lists.**
- **Each node has two part : data and address**

Let the two lists be:

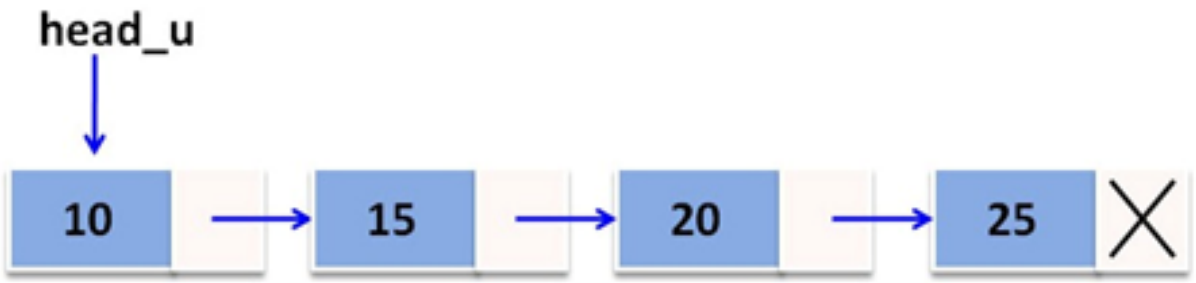


AND



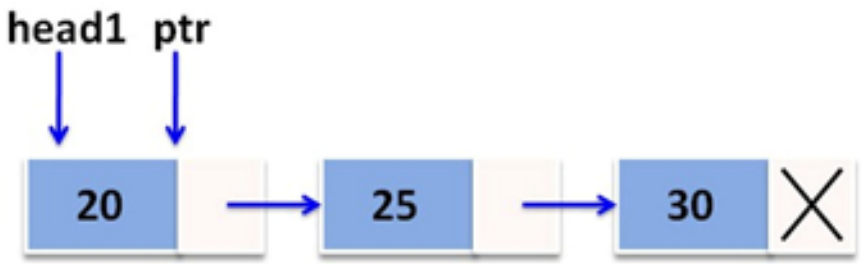
Union Operation on linked list

In the case of **union operation of two lists**, all the elements of the two lists will be in the 'union' list but the common elements of the two lists will appear just ones in the 'union'. **To perform union operation first the elements of the first list will be copied to the new list pointed by the pointer 'head_u' where the union is to be stored**. So, the list of the union will be:



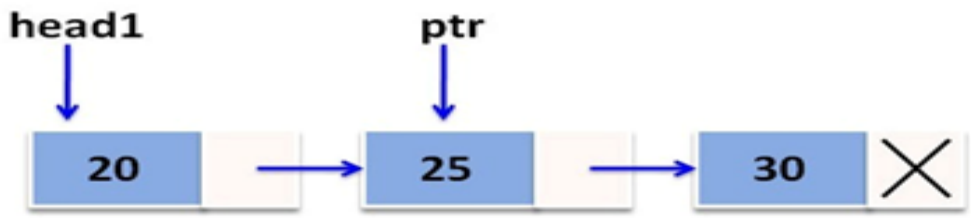
Union Operation on linked list

Now, a **pointer 'ptr' will be pointing to the first node** of the second list **pointed by 'head1'**.

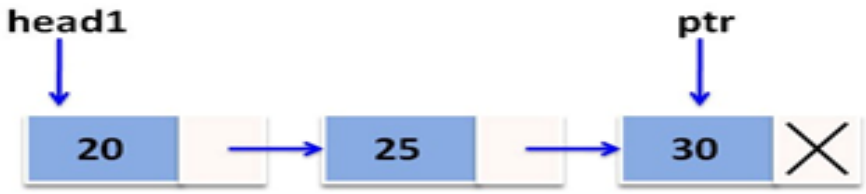


The **full list pointed by the pointer 'head_u'will be searched for the element pointed by 'ptr'** i.e. '20'. As the **data is already present** in the list storing the '**union**' then this **element will not be inserted in the 'union' list**. The pointer will be moved to the next node.

Union Operation on linked list



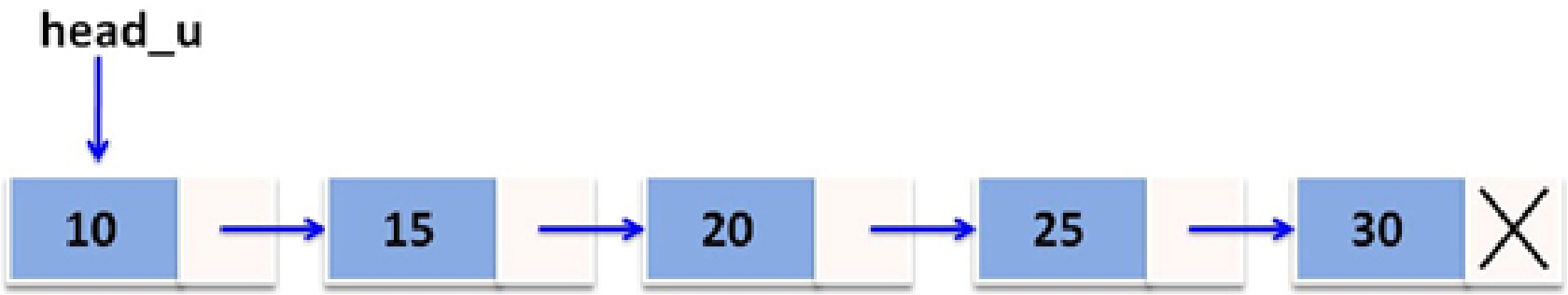
Again, the **same process will be followed**. The **data pointed** by '**ptr**' **will be compared with each element of the list pointed by 'head_u'**, as the data is already present in the union list then this data will also be **skipped** and the '**ptr**' will be moved to the next node.



Now **the same process will be continued** and as the **data (30) is not present in the list pointed by 'head_u' then the node with data (30)** will be **appended** to the list pointed by the pointer '**head_u**'.

Union Operation on linked list

Therefore, **the union of the two lists will be**:



intersection Operation on linked list

- **List 1 – 10,20,40,50**
- **List 2 – 20,50,70,60**

- **Output – 20,50**

Applications of linked list

- **Applications of linked list in computer science:**
- Implementation of stacks and queues
- Implementation of graphs: Adjacency list representation of graphs is the most popular which uses a linked list to store adjacent vertices.
- Dynamic memory allocation: We use a linked list of free blocks.
- Maintaining a directory of names
- Performing arithmetic operations on long integers
- Manipulation of polynomials by storing constants in the node of the linked list
- representing sparse matrices

Applications of linked list

- **Applications of linked list in the real world:**
- Image viewer – Previous and next images are linked and can be accessed by the next and previous buttons.
- Previous and next page in a web browser – We can access the previous and next URL searched in a web browser by pressing the back and next buttons since they are linked as a linked list.
- Music Player – Songs in the music player are linked to the previous and next songs. So you can play songs either from starting or ending of the list.
- GPS navigation systems- Linked lists can be used to store and manage a list of locations and routes, allowing users to easily navigate to their desired destination.
- Robotics- Linked lists can be used to implement control systems for robots, allowing them to navigate and interact with their environment.

Polynomial Manipulation

- Polynomial manipulations are one of the most important applications of linked lists.
- Polynomials are an important part of mathematics not inherently supported as a data type by most languages.
- A polynomial is a collection of different terms, each comprising coefficients, and exponents. It can be represented using a linked list.
- This representation makes polynomial manipulation efficient.
- While representing a polynomial using a linked list, each polynomial term represents a node in the linked list. To get better efficiency in processing, we assume that the term of every polynomial is stored within the linked list in the order of decreasing exponents. Also, no two terms have the same exponent, and no term has a zero coefficient and without coefficients. The coefficient takes a value of 1.

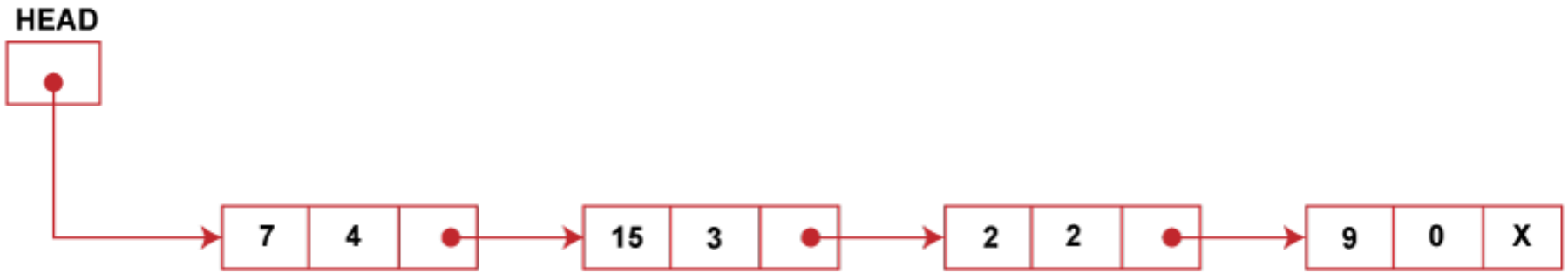
Polynomial Manipulation

- **Each node of a linked list representing polynomial constitute three parts:**
- The first part contains the value of the coefficient of the term.
- The second part contains the value of the exponent.
- The third part, LINK points to the next term (next node).
- **The structure of a node of a linked list that represents a polynomial is shown below:**

Node representing a term of a polynomial

Polynomial Manipulation

- Consider a polynomial $P(x) = 7x^4 + 15x^3 - 2x^2 + 9$. Here 7, 15, -2, and 9 are the coefficients, and 4,3,2,0 are the exponents of the terms in the polynomial. On representing this polynomial using a linked list, we have



Linked representation of polynomial P(x)

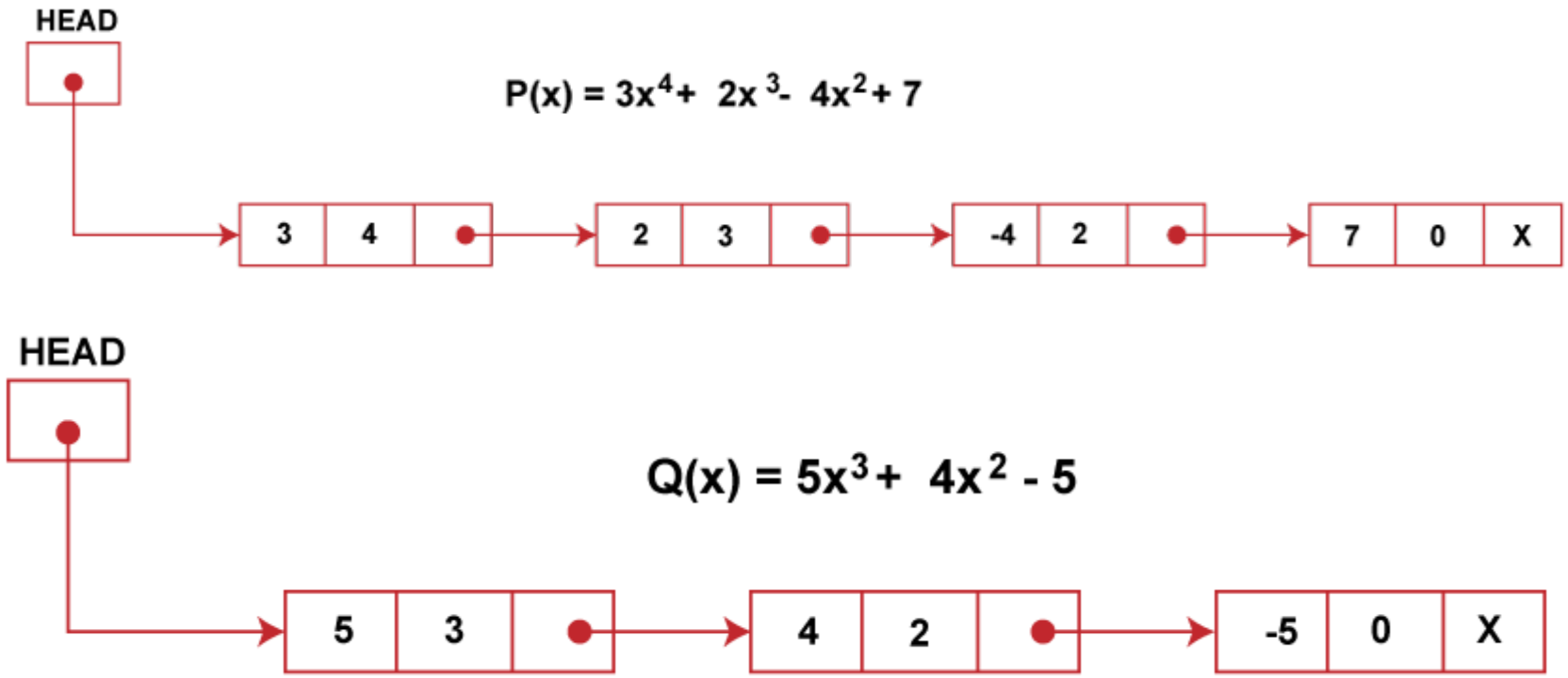
- Observe that the number of nodes equals the number of terms in the polynomial. So we have 4 nodes. Moreover, the terms are stored to decrease exponents in the linked list. Such representation of polynomial using linked lists makes the operations like subtraction, addition, multiplication, etc., on polynomial very easy.

Addition of two Polynomial

- To add two polynomials, we traverse the list P and Q. We take corresponding terms of the list P and Q and compare their exponents. If the two exponents are equal, the coefficients are added to create a new coefficient. If the new coefficient is equal to 0, then the term is dropped, and if it is not zero, it is inserted at the end of the new linked list containing the resulting polynomial. If one of the exponents is larger than the other, the corresponding term is immediately placed into the new linked list, and the term with the smaller exponent is held to be compared with the next term from the other list. If one list ends before the other, the rest of the terms of the longer list is inserted at the end of the new linked list containing the resulting polynomial.
- Let us consider an example an example to show how the addition of two polynomials is performed,
- $P(x) = 3x^4 + 2x^3 - 4x^2 + 7$ and $Q(x) = 5x^3 + 4x^2 - 5$

Addition of two Polynomial

- These polynomials are represented using a linked list in order of decreasing exponents as follows:



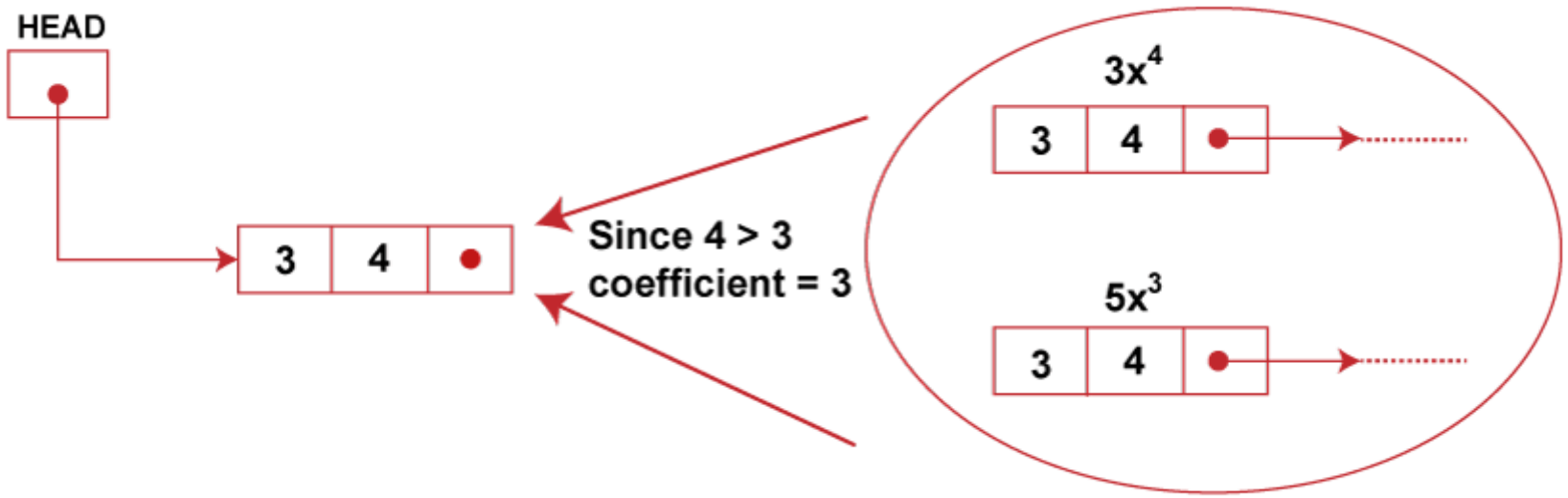
Addition of two Polynomial

- To generate a new linked list for the resulting polynomials that is formed on the addition of given polynomials P(x) and Q(x), we perform the following steps,

1. Traverse the two lists P and Q and examine all the nodes.

2. We compare the exponents of the corresponding terms of two polynomials. The first term of polynomials P and Q contain exponents 4 and 3, respectively. Since the exponent of the first term of the polynomial P is greater than the other polynomial Q, the term having a larger exponent is inserted into the new list. The new list initially looks as shown below:

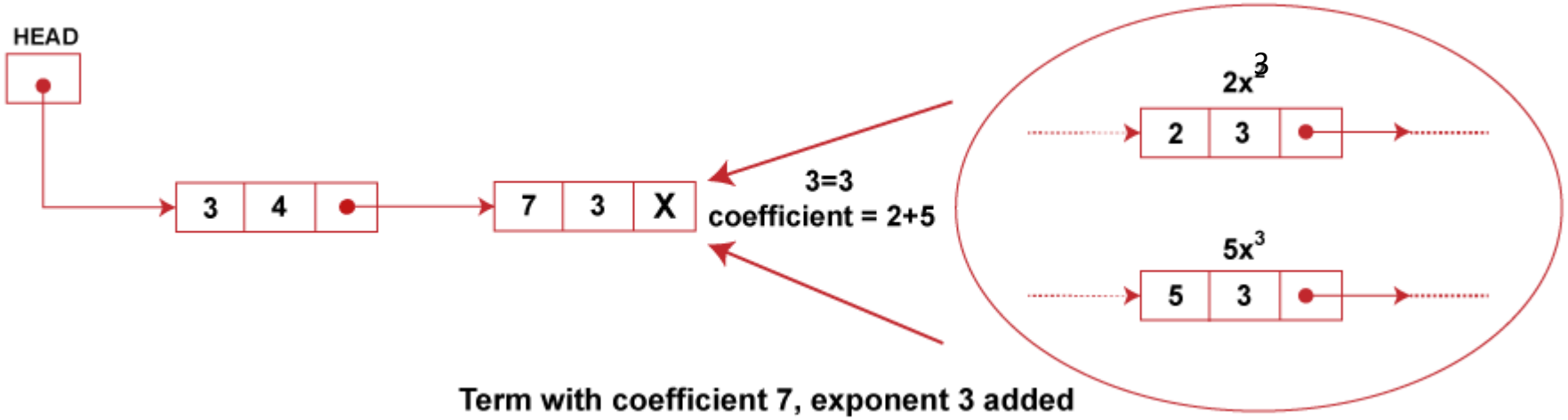
Addition of two Polynomial



Term with coefficient 3, exponent 4 added

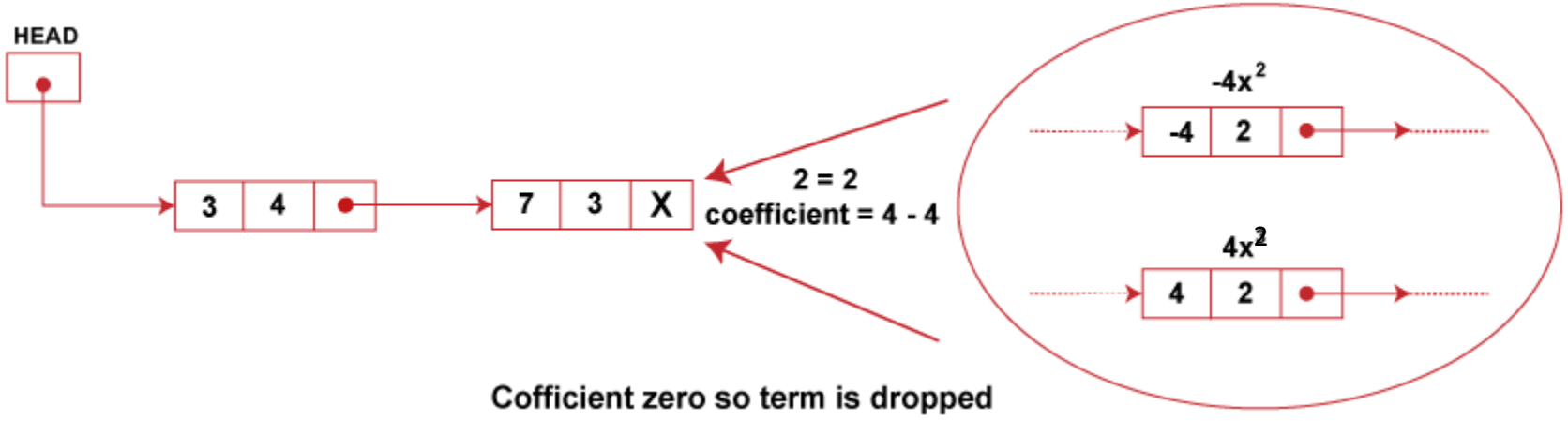
Addition of two Polynomial

- We then compare the exponent of the next term of the list P with the exponents of the present term of list Q. Since the two exponents are equal, so their coefficients are added and appended to the new list as follows:



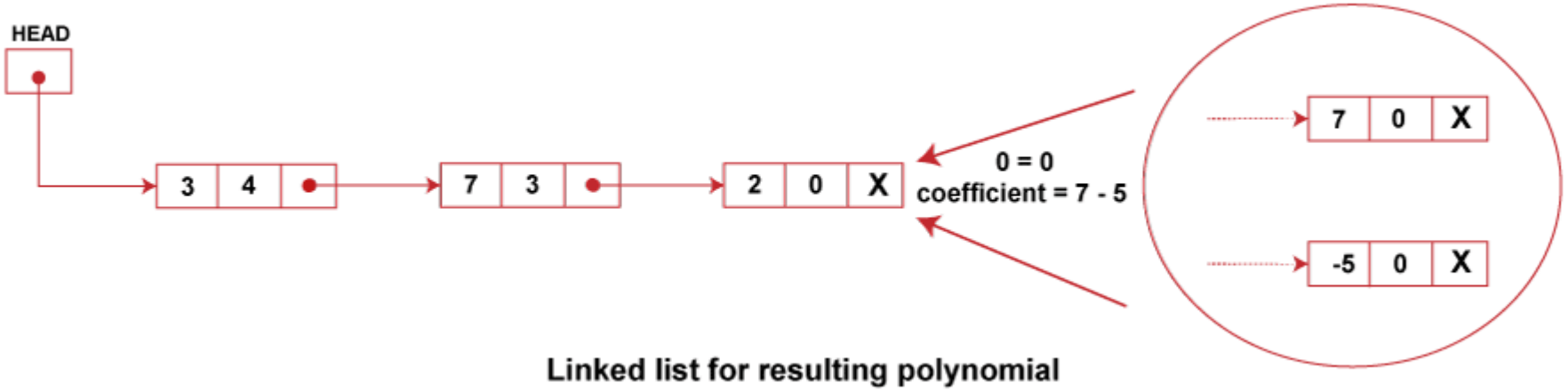
Addition of two Polynomial

- Then we move to the next term of P and Q lists and compare their exponents. Since exponents of both these terms are equal and after addition of their coefficients, we get 0, so the term is dropped, and no node is appended to the new list after this,



Addition of two Polynomial

- Moving to the next term of the two lists, P and Q, we find that the corresponding terms have the same exponents equal to 0. We add their coefficients and append them to the new list for the resulting polynomial as shown below:



Multiplication of two Polynomial

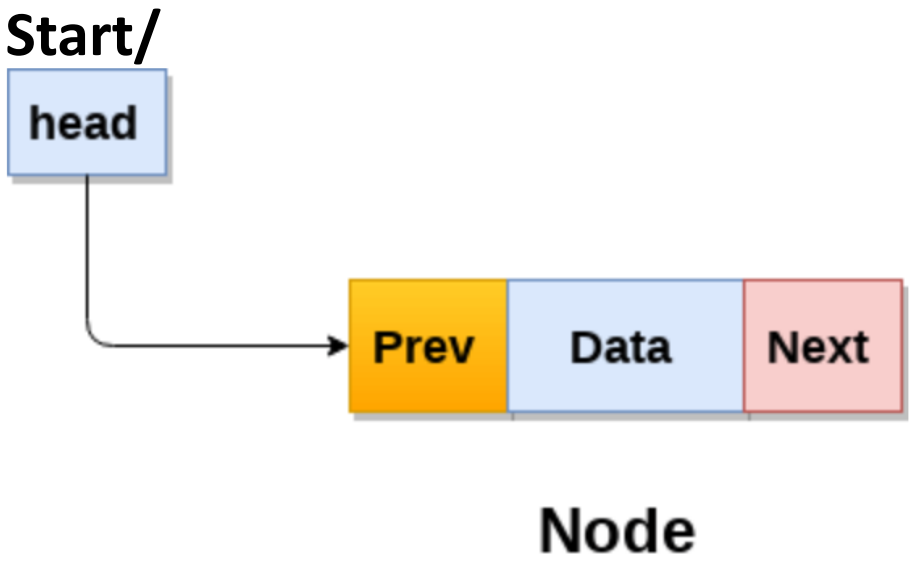
- We have been given two linked lists representing a polynomial each. Let's say **Poly1** and **Poly2** is the given polynomials, so we need to multiply the polynomials and print the output.
- Let's try to understand the problem with the help of examples by referring to the websites to learn to program.
- Suppose the given linked lists are:
  Poly1: $3x^3 + 6x^1 – 9$
  Poly2: $9x^3 – 8x^2 + 7x^1 + 2$
- Now, according to the problem statement, we need to multiply these polynomials **Poly1** and **Poly2**.
- So we will multiply each term in **Poly1** with every term in **Poly2**, and then we will add up all the terms with the same power of x such that each term in the final resultant polynomial list has a different power of x.
- **Output**
- Resultant Polynomial: $27x^6 – 24x^5 + 75x^4 – 123x^3 + 114x^2 – 51x^1 – 18$

Multiplication of two Polynomial

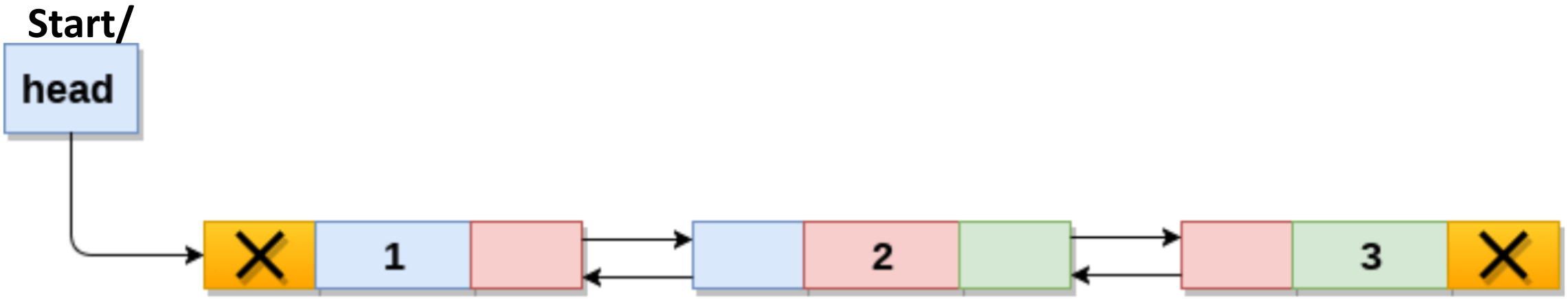
- **Some other examples:**
- **Input 1**
  Poly1: $6x^1 – 9$
  Poly2: $7x^1 + 2$
- **Output 1**
  Resultant Polynomial: $42x^2 – 51x^1 – 18$
- **Input 2**
  Poly1: $8x^1 + 7$
  Poly2: $4x^2 + 5$
- **Output 2**
  Resultant Polynomial: $32x^3 + 28x^2 + 40x^1 + 35$
-

Doubly Linked List

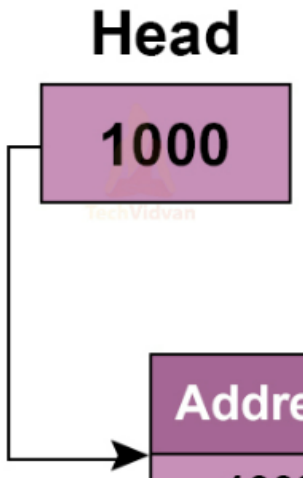
- Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence.
- Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer to the previous node (previous pointer).



Doubly Linked List



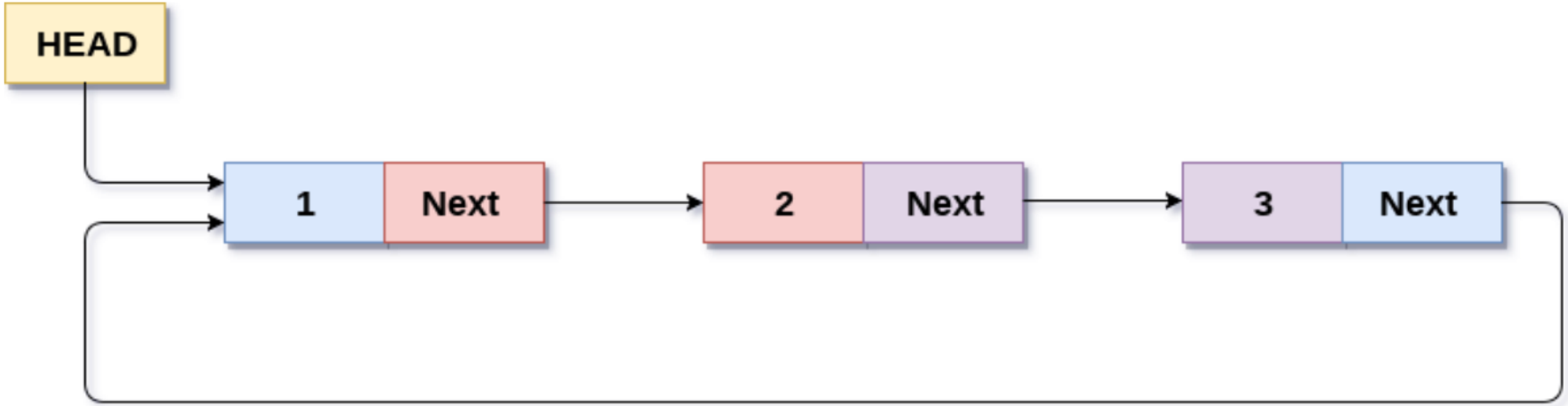
Doubly Linked List – memory representation



Address	Data	Prev	Next
1000	A	-1	2004
1056			
2004	B	1000	4002
2050			
3000			
4002	C	2004	6000
5060			
6000	D	4002	-1

Circular Linked List

- In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.
- We traverse a circular singly linked list until we reach the same node where we started. The circular singly liked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.



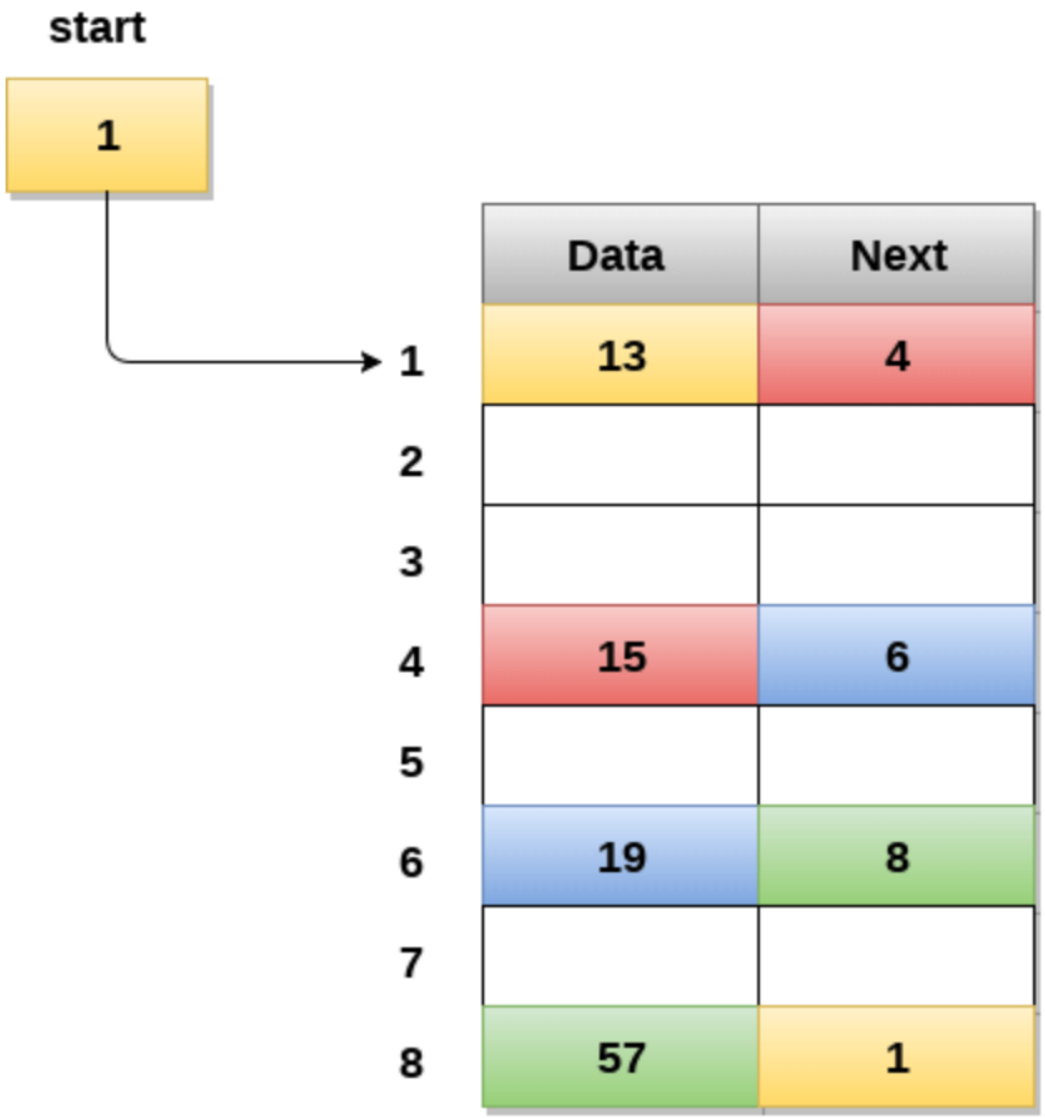
Circular Linked List

- Circular linked list are mostly used in task maintenance in operating systems. There are many examples where circular linked list are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

Memory Representation of circular linked list

- In the following image, memory representation of a circular linked list containing marks of a student in 4 subjects.
- However, the image shows a glimpse of how the circular list is being stored in the memory.
- The start or head of the list is pointing to the element with the index 1 and containing 13 marks in the data part and 4 in the next part.
- Which means that it is linked with the node that is being stored at 4th index of the list.
- However, due to the fact that we are considering circular linked list in the memory therefore the last node of the list contains the address of the first node of the list.

Memory Representation of circular linked list



Sparse matrix

- A matrix can be defined as a two-dimensional array having 'm' rows and 'n' columns. A matrix with m rows and n columns is called m × n matrix. It is a set of numbers that are arranged in the horizontal or vertical lines of entries.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Row (m)

Columns (n)

- What is a sparse matrix?
- Sparse matrices are those matrices that have the majority of their elements equal to zero. In other words, the sparse matrix can be defined as the matrix that has a greater number of zero elements than the non-zero elements.
- Now, the question arises: we can also use the simple matrix to store the elements, then why is the sparse matrix required?

Sparse matrix

- Why is a sparse matrix required if we can use the simple matrix to store elements?
- There are the following benefits of using the sparse matrix -
- **Storage -** We know that a sparse matrix contains lesser non-zero elements than zero, so less memory can be used to store elements. It evaluates only the non-zero elements.
- **Computing time:** In the case of searching in sparse matrix, we need to traverse only the non-zero elements rather than traversing all the sparse matrix elements. It saves computing time by logically designing a data structure traversing non-zero elements.
-

Sparse matrix

- Representation of sparse matrix
- Now, let's see the representation of the sparse matrix. The non-zero elements in the sparse matrix can be stored using triplets that are rows, columns, and values. There are two ways to represent the sparse matrix that are listed as follows -
- Array representation
- Linked list representation
-

Linked List representation of the sparse matrix

- In a linked list representation, the linked list data structure is used to represent the sparse matrix. The advantage of using a linked list to represent the sparse matrix is that the complexity of inserting or deleting a node in a linked list is lesser than the array.
- Unlike the array representation, a node in the linked list representation consists of four fields. The four fields of the linked list are given as follows -
- **Row -** It represents the index of the row where the non-zero element is located.
- **Column -** It represents the index of the column where the non-zero element is located.
- **Value -** It is the value of the non-zero element that is located at the index (row, column).
- **Next node -** It stores the address of the next node.

Linked List representation of the sparse matrix

- The node structure of the linked list representation of the sparse matrix is shown in the below image -
-

Node Structure

Row	Column	Value	Pointer to Next Node

- Let's understand the linked list representation of sparse matrix with the help of the example given below - Consider the sparse matrix -

Sparse Matrix →

	0	1	2	3
0	0	0	1	0
1	3	0	0	0
2	0	4	5	0
3	0	6	0	0

Linked List representation of the sparse matrix

- In the above figure, we can observe a 4x4 sparse matrix containing 5 non-zero elements and 11 zero elements. Above matrix occupies 4x4 = 16 memory space. Increasing the size of matrix will increase the wastage space.
- The linked list representation of the above matrix is given below –

Sparse Matrix

	0	1	2	3
0	0	0	1	0
1	3	0	0	0
2	0	4	5	0
3	0	6	0	0

- In the above figure, the sparse matrix is represented in the linked list form. In the node, the first field represents the index of the row, the second field represents the index of the column, the third field represents the value, and the fourth field contains the address of the next node.
- In the above figure, the first field of the first node of the linked list contains 0, which means 0th row, the second field contains 2, which means 2nd column, and the third field contains 1 that is the non-zero element. So, the first node represents that element 1 is stored at the 0th row-2nd column in the given sparse matrix. In a similar manner, all of the nodes represent the non-zero elements of the sparse matrix.

Linked List representation of the sparse matrix

If most of the elements in a matrix have the value 0, then the matrix is called spare matrix.

Example For 3 X 3 Sparse Matrix:

| 1 0 0 |
| 0 0 0 |
| 0 4 0 |

3-Tuple Representation Of Sparse Matrix Using Arrays:

| 3 3 2 |
| 0 0 1 |
| 2 1 4 |

Elements in the first row represents the number of rows, columns and non-zero values in sparse matrix.
First Row - | 3 3 2 |
3 - rows
3 - columns
2 - non- zero values

Elements in the other rows gives information about the location and value of non-zero elements.
| 0 0 1 | (Second Row) - represents value 1 at 0th Row, 0th column
| 2 1 4 | (Third Row) - represents value 4 at 2nd Row, 1st column